BioNAFNet: Biologically Inspired Low-Light Image Enhancement with Knowledge Distillation and Desktop Application

Submitted by

[Student Name 1] (BSSE-[Roll#]-[Section])

[Student Name 2] (BSSE-[Roll#]-[Section])

Session

[Write Session here]

Supervised by

[Teacher Name]



Department of Software Engineering

Lahore Garrison University

Lahore

COPYRIGHTS

This is to certify that the project titled "BioNAFNet: Biologically-Inspired Low-Light Image Enhancement with Knowledge Distillation and Desktop Application" is the genuine work carried out by [Student Names], student of BSSE of Software Engineering Department, Lahore Garrison University, Lahore. During the academic year [Year], in partial fulfilment of the requirements for the award of the degree of Bachelor of Science in Software Engineering and that the project has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

[Student Name 1] _____

[Student Name 2] _____

DECLARATION

This is to declare that the project entitled "BioNAFNet: Biologically-Inspired Low-Light Image Enhancement with Knowledge Distillation and Desktop Application" is an original work done by undersigned, in partial fulfilment of the requirements for the degree "Bachelor of Science in Software Engineering" at Software Engineering Department, Lahore Garrison University, Lahore. All the analysis, design and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.

[Student Name 1] _____

[Student Name 2] _____

ACKNOWLEDGEMENTS

We would like to thank our supervisor [Teacher Name] for guiding us throughout this project. Their knowledge of deep learning and computer vision helped us design and build BioNAFNet and the NeonEnhancer Pro desktop application.

We also thank Dr. Omer Irshad, Head of Department of Software Engineering, Lahore Garrison University, for providing us the resources and academic environment to complete this work.

We thank the creators of LOL datasets (Wei et al., 2018; Yang et al., 2021) for making their data available for research. We also thank the NAFNet authors (Chen et al., 2022) for their backbone architecture and the CustomTkinter community for the modern UI framework.

Finally, we thank our families and friends for their patience and support during the long training hours and late nights of coding.

DEDICATION

Dedicated to our families,
who supported us throughout this journey,
and to making machines see in the dark
just like the human eye.

Table of Contents

Chapter 1 16

Introduction 16

1.1 Background 16

1.2 Motivation 17

1.3 Problem Statement 17

1.4 Research Objectives 17

1.5 Scope of the Project 18

1.6 Novel Contributions 18

1.7 Significance 19

1.8 Project Timeline / Gantt Chart 19

1.9 Report Organization 19

Chapter 2 21

Problem Definition 21

2.1 Problem Overview 21

2.2 Types of Low-Light Degradation 21

2.3 Limitations of Existing Solutions 22

2.4 Formal Problem Statement 22

2.5 Research Questions 22

Chapter 3 24

Software Requirement Specification 24

3.1 Literature Survey 24

3.2 Research Gaps 26

3.3 Benchmark Datasets 27

3.4 Functional Requirements 27

3.5 Non-Functional Requirements 28

3.6 Hardware and Software Requirements 29

3.7 Evaluation Metrics 29

Chapter 4 30
Methodology 30
4.1 Research Approach 30
4.2 NAFNet Backbone 30
4.3 Module 1: RLIN (Retinal Lateral Inhibition Network) 30
4.4 Module 2: FAEB (Frequency-Aware Enhancement Branch) 33
4.5 Module 3: ARFP (Adaptive Receptive Field Pooling) 33
4.6 Module 4: PRCM (Photoreceptor Response Curve Modeling) 33
4.7 Module 5: SPDP (Scotopic-Photopic Dual Pathway) 34
4.8 Module 6: TAM (Temporal Adaptation Memory) 34
4.9 Module 7: WFPL (Weber-Fechner Perceptual Loss) 34
4.10 Loss Function Design 35
4.11 Training Strategy 35
4.12 Knowledge Distillation Pipeline 36

4.13 Ensemble Fusion Strategy 37
Chapter 5 38
System Architecture / Initial Design 38
5.1 System Architecture 38
5.2 NeonEnhancer Pro Application Architecture 40
5.3 Detailed System Design 42
Chapter 6 50
Implementation and Testing 50
6.1 Implementation Overview 50
6.2 Code Structure 50
6.3 Key Implementation Details 51
6.4 NeonEnhancer Pro Application Features 52
6.5 Training Process 53
6.6 Testing Methodology 54
6.7 Unit Testing 54
6.8 Integration Testing 55

6.9 Software Quality 56
Chapter 7 57
Results and Discussion 57
7.1 Quantitative Results Overview 57
7.2 Complete PSNR Benchmark 57
7.3 Complete SSIM Benchmark 57
7.4 Comparison with State-of-the-Art 58
7.5 Knowledge Distillation Results 58
7.6 Ensemble Fusion Results 59
7.7 Ablation Study 59
7.8 Speed vs Quality Analysis 60
7.9 Training Convergence Analysis 61
7.10 Qualitative Results 61
7.11 Discussion 64
Chapter 8 66
Conclusion and Future Work 66
8.1 Summary of Work 66
8.2 Key Achievements 66
8.3 Contributions to Knowledge 67
8.4 Limitations 67
8.5 Future Work 68
8.6 Conclusion 69
References 70
Appendices 73
Appendix A: Complete Network Configuration 73
Appendix B: Training Log Summary 73
Appendix C: Loss Function Weights 73
Appendix D: Per-Dataset Results Breakdown 74
Appendix E: BioNAFBlock Code Snippet 74

Appendix F: Project Timeline 75

List of Tables

Table 1: List of Abbreviations 14
Table 2: Project Gantt Chart / Timeline 19
Table 3: Summary of Traditional Enhancement Methods 24
Table 4: Performance Comparison with State-of-the-Art Methods on LOLv1 Dataset 26
Table 5: Benchmark Datasets for Low-Light Image Enhancement 27
Table 6: Functional Requirements 27
Table 7: Non-Functional Requirements 28
Table 8: Hardware and Software Requirements 29
Table 9: Composite Loss Function — Teacher Training 35
Table 10: Distillation Loss Function — Student Training 35
Table 11: Complete Training Configuration 35
Table 12: Ensemble Fusion Strategies 37
Table 13: BioNAFNet Layer-by-Layer Architecture 40
Table 14: NeonEnhancer Pro Application Pages 40
Table 15: All 12 Supported Enhancement Models 41
Table 16: System Component Hierarchy 42
Table 17: Project Code File Structure 50
Table 18: Test Set Details 54
Table 19: Unit Test Results 54
Table 20: Integration Test Results 55

Table 21: Complete PSNR Benchmark Results (Full-Resolution Test Images) 57
Table 22: Complete SSIM Benchmark Results 57
Table 23: Comprehensive Comparison with State-of-the-Art Methods on LOLv1 58
Table 24: Knowledge Distillation: Teacher vs Student Detailed Comparison 58
Table 25: Ensemble Fusion Strategies — Detailed Results 59
Table 26: Ablation Study — Estimated Module Contributions 59
Table 27: Ablation Study — Impact of Removing Individual Modules 60
Table 28: Inference Speed Comparison 60
Table 29: Complete Network Hyperparameters 73

Table 30: Training Log Summary Statistics 73
Table 31: Complete Loss Function Configuration 73
Table 32: Per-Dataset PSNR Distribution 74
Table 33: Detailed Project Week-by-Week Timeline 75

List of Figures

Figure 1: NeonEnhancer Pro Main Window — Dark Futuristic Theme with Enhancement Page 20
Figure 2: Low-Light Input vs BioNAFNet Enhanced Output vs Ground Truth 22
Figure 3: Side-by-Side Comparison: Input (with annotations for noise, color shift, detail loss) | BioNAFNet Enhanced | Ground Truth 23
Figure 4: RLIN Module Architecture — Center-Surround Pathways with Subtraction and Scaling 33
Figure 5: FAEB Module — FFT → Amplitude/Phase Processing → iFFT → Blend 33
Figure 6: Knowledge Distillation Pipeline — Teacher (154M) to Student (24M) 37
Figure 7: BioNAFNet System Architecture Block Diagram — Input → PRCM → SPDP → Encoder+RLIN → FAEB → Decoder → ARFP → Output 40
Figure 8: NeonEnhancer Pro Enhance Page — Model Selection Dropdown with 12 Options 41
Figure 9: Use Case Diagram — User interactions: Upload Image, Select Model, Enhance, View Gallery, Batch Process, Augment, Crop, Resize, Live Preview 44
Figure 10: Class Diagram — NeonEnhancerPro contains 5 pages; BioNAFNet contains BioNAFBlock, LateralInhibition, FrequencyBranch, ARFP, PRCM, SPDP, TAM 44
Figure 11: Activity Diagram — Start → Load Image → Normalize → Pad → PRCM → SPDP → Encoder → FAEB → Decoder → ARFP → Residual → Crop → Clamp → Save → End 46
Figure 12: Sequence Diagram — User → NeonEnhancerPro: click Enhance → Thread: run model → BioNAFNet: forward() → return PIL Image → update gallery 47
Figure 13: Data Flow Diagram (Level 0) — User sends images to NeonEnhancer Pro, outputs to Enhanced Images directory 48

Figure 14: Enhancement Page — After processing a batch with BioNAFNet Student, showing before/after gallery (1 / 15) 52
Figure 15: Augmentation Page — Slider set to 3 variations with augmented image preview 52
Figure 16: Cropping Page — Image with crop region highlighted and preset buttons visible 52

Figure 17: Resizing Page — Width/height sliders with aspect ratio lock toggle 53
Figure 18: Live Preview Page — Webcam feed (left) vs Enhanced version (right) with FPS counter 53
Figure 19: Teacher Model Training Curves — PSNR and Loss Over 504 Epochs 53
Figure 20: Teacher Model — Early vs Late Training Convergence 53
Figure 21: Student Model Distillation Training Curves — PSNR and Loss Over 693 Epochs 53
Figure 22: Training Loss Comparison: Teacher vs Student 54
Figure 23: Teacher vs Student — Validation PSNR Convergence 54
Figure 24: Teacher vs Student — Validation SSIM Over Training 54
Figure 25: Learning Rate Schedule for Both Models (CosineAnnealingWarmRestarts) 54
Figure 26: PSNR Comparison Bar Chart — All 6 Models Across 3 Datasets 57
Figure 27: SSIM Comparison Bar Chart — All 6 Models Across 3 Datasets 57
Figure 28: PSNR Performance Heatmap — Models vs Datasets 57
Figure 29: PSNR Comparison with State-of-the-Art Methods on LOLv1 (Bar Chart) 58
Figure 30: Knowledge Distillation Compression Metrics — Visual Summary 59
Figure 31: Ensemble Fusion Strategies — PSNR Comparison Chart 59
Figure 32: Ablation Study — Incremental Contribution of Each Biological Module 60
Figure 33: Speed vs Quality Tradeoff — Inference Time vs PSNR for Each Model 61

Figure 34: Model Capability Profiles — Teacher, Student, and Ensemble 61
Figure 35: Model Size Comparison — Parameter Count and File Size 61
Figure 36: Train-Validation PSNR Gap — Teacher and Student Models 61
Figure 37: Teacher Model PSNR Convergence Detail 61
Figure 38: Visual Comparison Grid (5 columns × 4 rows): Input | RetinexNet | Retinexformer | BioNAFNet (Ours) | Ground Truth 62
Figure 39: Zoomed Crop Comparison: Input | Teacher | Student | Ensemble | Ground Truth 62
Figure 40: Enhancement Examples for LOLv2 Synthetic: 3 pairs (Input | BioNAFNet Enhanced) 62
Figure 41: NeonEnhancer Pro Complete Enhancement Workflow — Upload, Select Model, Enhance, View Gallery 63

List of Abbreviations

Table 1: List of Abbreviations

Abbreviation	Full Form
LLIE	Low-Light Image Enhancement
CNN	Convolutional Neural Network
GAN	Generative Adversarial Network
NAFNet	Nonlinear Activation Free Network
BioNAFNet	Biologically-Inspired NAFNet
RLIN	Retinal Lateral Inhibition Network
FAEB	Frequency-Aware Enhancement Branch
ARFP	Adaptive Receptive Field Pooling
PRCM	Photoreceptor Response Curve Modeling
SPDP	Scotopic-Photopic Dual Pathway
TAM	Temporal Adaptation Memory
WFPL	Weber-Fechner Perceptual Loss
PSNR	Peak Signal-to-Noise Ratio
SSIM	Structural Similarity Index Measure
EMA	Exponential Moving Average
AMP	Automatic Mixed Precision
GPU	Graphics Processing Unit
FPS	Frames Per Second
FFT	Fast Fourier Transform
GRU	Gated Recurrent Unit
SCA	Simplified Channel Attention
VGG	Visual Geometry Group Network
LOL	Low-Light Dataset
KD	Knowledge Distillation
LR	Learning Rate
UI	User Interface
CTk	CustomTkinter
RGB	Red Green Blue Color Space
YUV	Luminance-Chrominance Color Space
CLAHE	Contrast Limited Adaptive Histogram Equalization
UNet	U-shaped Network Architecture
DL	Deep Learning

Abstract

Low-light image enhancement is a real problem that affects security cameras, mobile photography, and many other areas where pictures are taken in dark conditions. Images captured in low light have noise, wrong colors, and missing details. Traditional methods like histogram equalization can brighten images but they also make the noise worse. Deep learning methods have shown good results but none of them use the biological mechanisms that allow the human eye to adapt to darkness.

In this project, we built two things: first, a deep learning model called BioNAFNet that enhances low-light images using ideas from how the human eye works; second, a desktop application called NeonEnhancer Pro built with CustomTkinter that lets users enhance their images through a simple and modern interface.

BioNAFNet integrates seven biologically-inspired modules into the NAFNet backbone: Retinal Lateral Inhibition Network (RLIN) for edge enhancement, Frequency-Aware Enhancement Branch (FAEB) for spectral recovery, Adaptive Receptive Field Pooling (ARFP) for luminance-dependent processing, Photoreceptor Response Curve Modeling (PRCM) for rod and cone simulation, Scotopic-Photopic Dual Pathway (SPDP) for separate luminance and color processing, Temporal Adaptation Memory (TAM) for iterative refinement, and Weber-Fechner Perceptual Loss (WFPL) for perception-weighted training.

We trained a big Teacher model (154 million parameters) and then compressed it into a small Student model (24 million parameters) through knowledge distillation. The Student is 6.4 times smaller but still gets great results. On the LOLv1 test set, the Student gets 25.23 dB PSNR which is actually higher than the Teacher (24.94 dB). When we combine both models together using ensemble fusion, we get the best result of 29.29 dB on the LOLv2 Real dataset. The Student model runs at 20+ frames per second which makes real-time enhancement possible.

The NeonEnhancer Pro desktop application has five main features: image enhancement with 12 different AI models, data augmentation, image cropping, image resizing, and live webcam preview. The application uses a futuristic dark theme and provides a complete tool for anyone working with low-light images.

Chapter 1

Introduction

1.1 Background

Image enhancement is one of the oldest and most important tasks in computer vision. The idea is simple: take a bad image and make it look better. But when it comes to low-light images specifically, things get much more complicated than just making it brighter. Low-light images have multiple problems happening at the same time: they are dark, noisy, color-shifted, and lacking in detail. Fixing one problem often makes another worse.

Traditional methods like Histogram Equalization (1970s) and Gamma Correction can make images brighter, but they have a big limitation. They apply the same mathematical formula to every pixel without understanding what is actually in the image. So when they boost brightness, they also boost noise. When they fix contrast, they sometimes create ugly artifacts. These methods simply cannot handle the complexity of real low-light degradation.

Deep learning changed the game. Instead of writing rules by hand, we can now train neural networks on thousands of example pairs (dark image paired with its well-lit version) and the network learns how to do the enhancement automatically. Models like RetinexNet, MIRNet, and Retinexformer have achieved impressive results. But there is something interesting that nobody has tried: using the biological mechanisms of the human eye as inspiration for the neural network design.

Our eyes can see in conditions ranging from bright sunlight to near total darkness. This is possible because of specific biological mechanisms like photoreceptor cells (rods and cones), lateral inhibition between neurons, and temporal adaptation. In this project, we take these biological principles and implement them as neural network modules inside a model called BioNAFNet.

1.2 Motivation

The human visual system is remarkable. Our eyes can adapt to lighting conditions spanning over 10 orders of magnitude, from starlight to bright sunshine. This adaptation happens through specific biological mechanisms: rod and cone photoreceptors with different sensitivity curves, lateral inhibition between retinal neurons that enhances edges, separate processing pathways for brightness and color, and temporal adaptation that gradually adjusts sensitivity over time.

These mechanisms have been refined by millions of years of evolution. It makes sense to ask: can we put these biological principles into our deep learning models to make them better at low-light enhancement? That is exactly what BioNAFNet does. We take the NAFNet architecture, which is already a strong backbone for image restoration, and we add seven modules that are each inspired by a specific biological mechanism of the human visual system.

1.3 Problem Statement

Low-light images have four main problems happening at the same time: (1) noise from the camera sensor, (2) colors that look wrong, (3) loss of fine details like text and edges, and (4) very low contrast where everything looks flat. Existing deep learning methods can fix these problems to some extent, but they do not use the biological mechanisms that make the human eye so good at adapting to dark conditions. Also, most good models are too slow for real-time use and there is no existing desktop application that combines multiple enhancement models into one easy-to-use tool.

1.4 Research Objectives

1. Design a novel architecture called BioNAFNet with seven biologically-inspired modules (RLIN, FAEB, ARFP, PRCM, SPDP, TAM, WFPL) integrated into the NAFNet backbone.
2. Train the full Teacher model (154M parameters) on LOLv1 and LOLv2 datasets and achieve competitive results with state-of-the-art methods.
3. Compress the Teacher into a lightweight Student model (24M parameters) using knowledge distillation for real-time deployment at 20+ FPS.
4. Demonstrate ensemble fusion of Teacher and Student outputs for maximum enhancement quality.

5. Build a modern desktop application (NeonEnhancer Pro) using CustomTkinter with image enhancement, augmentation, cropping, resizing, and live webcam preview.

6. Evaluate all model variants on LOLv1, LOLv2 Real, and LOLv2 Synthetic test sets using PSNR and SSIM metrics.

1.5 Scope of the Project

This project covers single-image low-light enhancement using supervised learning with paired datasets. We use three benchmark datasets: LOLv1 (485/15 train/test pairs), LOLv2 Real (689/100), and LOLv2 Synthetic (900/100). The desktop application supports 12 different enhancement models (3 BioNAFNet variants plus 9 other published methods), data augmentation, cropping, resizing, and live webcam enhancement. Training and testing are done on a standard desktop with an NVIDIA GPU. Everything runs on a local machine without needing the internet.

1.6 Novel Contributions

1. RLIN (Retinal Lateral Inhibition Network): Center-surround edge enhancement inspired by retinal ganglion cell receptive fields.

2. FAEB (Frequency-Aware Enhancement Branch): Fourier-domain amplitude and phase correction at the network bottleneck.

3. ARFP (Adaptive Receptive Field Pooling): Multi-scale convolution (3x3, 5x5, 7x7) with luminance-dependent gating.

4. PRCM (Photoreceptor Response Curve Modeling): Learned rod and cone gamma curves with adaptive scotopic-photopic threshold.

5. SPDP (Scotopic-Photopic Dual Pathway): Separate luminance and chrominance processing inspired by magnocellular and parvocellular pathways.

6. TAM (Temporal Adaptation Memory): GRU-based iterative refinement across multiple forward passes, modeling dark adaptation.

7. WFPL (Weber-Fechner Perceptual Loss): Perception-weighted loss emphasizing dark region accuracy based on psychophysical principles.

8. Knowledge distillation pipeline achieving 6.4x compression with the Student actually beating the Teacher on LOLv1.

9. NeonEnhancer Pro: a full desktop application with 5 pages and 12 model options built using CustomTkinter.

1.7 Significance

From a theoretical perspective, BioNAFNet is the first architecture to systematically integrate multiple biological visual processing mechanisms into a single deep learning model for low-light image enhancement. Each module has a clear biological analog which makes the architecture interpretable and scientifically grounded.

From a practical perspective, the Student model (BioNAFNet-Lite) at 24M parameters and 20+ FPS enables real-time low-light enhancement on standard desktop GPUs. The ensemble fusion strategy provides the highest quality for offline processing. The NeonEnhancer Pro application makes this technology accessible to regular users through a modern, intuitive interface.

1.8 Project Timeline / Gantt Chart

Table 2: Project Gantt Chart / Timeline

Week	Activity	Deliverable
1-2	Literature review, topic selection	Proposal draft
3-4	NAFNet study, dataset preparation	Dataset ready
5-6	BioNAFNet design, RLIN, FAEB coding	Architecture code
7-8	Teacher training (504 epochs)	Trained Teacher model
9-10	Knowledge distillation (693 epochs)	Trained Student model
11-12	Ensemble testing, benchmarking	Results tables
13-14	NeonEnhancer Pro UI development	Working application
15-16	Testing, documentation, final report	This document

1.9 Report Organization

- Chapter 2 defines the problem of low-light image enhancement and explains the challenges.
- Chapter 3 covers the software requirements, literature survey, and research gaps.
- Chapter 4 describes our methodology including all seven biological modules, loss functions, and training strategy.
- Chapter 5 presents the system architecture, detailed design, and UML diagrams.
- Chapter 6 covers implementation details, application features, and testing procedures.

- Chapter 7 presents all experimental results, benchmark comparisons, and discussion.
- Chapter 8 provides conclusions and future work directions.

[INSERT FIGURE 1 HERE]

Figure 1: NeonEnhancer Pro Main Window — Dark Futuristic Theme with Enhancement Page

Chapter 2

Problem Definition

2.1 Problem Overview

Low-light image enhancement is the task of recovering a high-quality, well-lit image from a degraded input captured in poor lighting conditions. When a camera captures an image in low light, multiple things go wrong at once. The sensor receives very few photons, so the electronic noise becomes a big part of the signal. The auto white balance fails and creates unnatural color shifts. Fine textures and edges become invisible because pixel values are very close to zero. The overall contrast is compressed into a very narrow range.

The challenge is not just making the image brighter. Any simple brightening method also amplifies the noise. The real challenge is to increase brightness, fix colors, recover details, and remove noise all at the same time without introducing new artifacts.

2.2 Types of Low-Light Degradation

2.2.1 Noise Amplification

When the camera sensor receives very few photons in dark conditions, the electronic noise from the sensor circuitry becomes significant relative to the actual signal. This noise appears as random grain throughout the image and is especially bad in the darkest regions. Any attempt to brighten the image also amplifies this noise.

2.2.2 Color Distortion

Camera auto white balance algorithms are designed for normal lighting. In low light, they often fail completely, producing yellowish, greenish, or bluish tints that make the image look unnatural. Skin tones look wrong and environmental colors are shifted.

2.2.3 Loss of Fine Details

When pixel intensity values are very close to zero, the difference between adjacent pixels becomes smaller than the noise level. This means fine textures, small text, and subtle edges are genuinely lost. They are not just hidden, they are gone from the captured data.

2.2.4 Low Dynamic Range

In a well-lit scene, pixel values span from 0 to 255. In a low-light image, most pixels are between 0 and 30. Everything looks flat and lifeless because there is no contrast between different parts of the scene.

[INSERT FIGURE 2 HERE]

Figure 2: Low-Light Input vs BioNAFNet Enhanced Output vs Ground Truth

2.3 Limitations of Existing Solutions

2.3.1 Traditional Methods

Histogram Equalization (HE) redistributes pixel intensities across the full range. It improves contrast but amplifies noise and often produces over-saturated results. CLAHE applies HE locally but creates blocky artifacts. Gamma correction uses a power-law transformation but cannot separate signal from noise. Retinex methods decompose images into reflectance and illumination but produce halo artifacts around edges.

2.3.2 Deep Learning Methods

Existing deep learning approaches like RetinexNet, KinD, MIRNet, and Retinexformer achieve much better results. However they share a common limitation: none of them incorporate the biological mechanisms that allow the human eye to handle extreme lighting variations. They treat low-light enhancement as a pure signal processing problem rather than drawing on the principles of biological vision.

2.3.3 The Quality-Speed Gap

Large models with many parameters (50M+) achieve good quality but are too slow for real-time applications. Lightweight models are fast but sacrifice quality. There is a gap between what is achievable in terms of quality and what is practical for deployment. Knowledge distillation can help bridge this gap but has not been widely applied to LLIE methods. Also, there is no existing desktop application that combines multiple AI models with image processing tools in one package.

2.4 Formal Problem Statement

"How can we design a deep learning architecture for low-light image enhancement that (a) incorporates biological visual processing principles to improve enhancement quality, (b) achieves competitive performance with state-of-the-art methods, (c) can be compressed into a lightweight model suitable for real-time deployment, and (d) is integrated into a user-friendly desktop application?"

2.5 Research Questions

1. RQ1: Can biologically-inspired modules improve the performance of deep learning models for low-light image enhancement?

2. RQ2: What is the individual contribution of each biological module to the overall enhancement quality?

3. RQ3: Can knowledge distillation effectively compress a large Teacher model into a real-time capable Student model?

4. RQ4: Does ensemble fusion of Teacher and Student produce results superior to either model alone?

[INSERT FIGURE 3 HERE]

Figure 3: Side-by-Side Comparison: Input (with annotations for noise, color shift, detail loss) | BioNAFNet Enhanced | Ground Truth

Chapter 3

Software Requirement Specification

3.1 Literature Survey

3.1.1 Traditional Image Enhancement Methods

Histogram Equalization (HE) was one of the earliest enhancement techniques, proposed in the 1970s. It maps pixel intensities to a uniform distribution, spreading values across the full 0-255 range [Gonzalez and Woods, 2002]. While it improves contrast, it also boosts noise and washes out regions in low-light images.

Contrast Limited Adaptive Histogram Equalization (CLAHE), introduced by Zuiderveld in 1994, improved upon HE by dividing the image into small tiles and applying local histogram equalization with a clip limit to prevent over-amplification. CLAHE is widely used in medical imaging but still creates blocking artifacts at tile boundaries and cannot handle really dark images well.

Gamma correction applies a power-law transformation (I_out = I_in^gamma) where gamma < 1 brightens the image. Adaptive gamma methods estimate gamma locally but still cannot tell the difference between signal and noise [Huang et al., 2013].

The Retinex theory, proposed by Edwin Land in 1977, says that perceived color depends on relative reflectance rather than absolute light intensity. Image formation is modeled as $I = R \times L$ where R is reflectance and L is illumination. Single Scale Retinex (SSR) and Multi-Scale Retinex (MSR) estimate illumination using Gaussian filters [Jobson et al., 1997]. While theoretically sound, Retinex methods produce halo artifacts and struggle with strong noise.

Table 3: Summary of Traditional Enhancement Methods

Method	Year	Approach	Pros	Cons
HE	1970s	Global histogram redistribution	Simple, fast	Over-enhancement, noise amplification
CLAHE	1994	Local adaptive HE	Better local contrast	Blocking artifacts, limited in dark
Gamma Correction	Various	Power-law transform	Simple, adjustable	Cannot denoise, fixed rule
SSR/MSR	1997	Retinex decomposition	Theoretically grounded	Halo artifacts, noise sensitive
LIME	2017	Illumination	Structure	Over-smoothing,

		estimation	preserving	slow

3.1.2 CNN-Based Enhancement Methods

LLNet (Lore et al., 2017) was one of the first deep learning approaches for LLIE, using stacked sparse denoising autoencoders. It showed that neural networks can learn enhancement from data, but its quality was limited by the simple architecture.

MBLLEN (Lv et al., 2018) used a Multi-Branch architecture that processes features at multiple scales through separate branches. This multi-scale approach helped capture both local and global features but increased computational cost.

RetinexNet (Wei et al., 2018) was one of the first to apply deep learning to LLIE using a Retinex decomposition network. KinD (Zhang et al., 2019) improved on this with a two-branch architecture achieving 20.87 dB on LOLv1.

MIRNet (Zamir et al., 2020) introduced multi-scale residual blocks with selective kernel features, reaching 24.14 dB. NAFNet (Chen et al., 2022) simplified things by removing activation functions entirely, using a SimpleGate mechanism instead. NAFNet is our backbone architecture because of its clean, efficient design.

3.1.3 Transformer-Based Enhancement Methods

Restormer (Zamir et al., 2022) applied efficient transformers to image restoration using multi-DConv head transposed attention, achieving state-of-the-art results across multiple restoration tasks.

SNR-Aware Transformer (Xu et al., 2022) introduced signal-to-noise ratio aware attention that adapts processing based on local noise levels. It achieved 24.61 dB on LOLv1.

Retinexformer (Cai et al., 2023) combined Retinex decomposition with transformer architecture, achieving 25.16 dB on LOLv1. It is currently one of the top-performing methods on LOL benchmarks.

3.1.4 GAN-Based Enhancement Methods

EnlightenGAN (Jiang et al., 2021) was notable for using unpaired training data through adversarial learning. While it removed the need for paired data, its results (17.48 dB) were lower than supervised methods.

3.1.5 Knowledge Distillation in Image Processing

Knowledge Distillation (KD), introduced by Hinton et al. (2015), compresses a large Teacher model into a smaller Student by training the Student to copy the Teacher outputs. KD has been

successfully applied in classification and detection tasks but it is not common in LLIE. Our work shows that KD can effectively compress a LLIE model by 6.4x while the Student actually beats the Teacher on some datasets.

Table 4: Performance Comparison with State-of-the-Art Methods on LOLv1 Dataset

Method	Year	Type	Params	PSNR (dB)	SSIM
LLNet	2017	Autoencoder	~1M	~17.0	~0.600
RetinexNet	2018	CNN	0.8M	16.77	0.560
MBLLEN	2018	Multi-Branch	~0.5M	17.90	0.715
KinD	2019	Retinex-CNN	8.2M	20.87	0.800
DRBN	2020	Recursive	5.3M	20.13	0.830
MIRNet	2020	Multi-Scale	31.8M	24.14	0.830
EnlightenGAN	2021	GAN	8.6M	17.48	0.650
SNR-Aware	2022	Transformer	39.1M	24.61	0.842
NAFNet	2022	CNN (SimpleGate)	67.9M	~23.0	~0.830
Restormer	2022	Transformer	26.1M	22.43	0.823
Retinexformer	2023	Retinex-Trans	1.6M	25.16	0.845
BioNAFNet-Lite (Ours)	2026	Bio-CNN	24M	25.23	0.853
BioNAFNet-Ens (Ours)	2026	Bio-CNN	178M	25.27	0.864

3.2 Research Gaps

From the literature survey, we found the following gaps:

1. Gap 1 — No Biological Integration: No existing LLIE method uses biological visual system mechanisms like photoreceptor modeling, lateral inhibition, or temporal adaptation.

2. Gap 2 — Limited Frequency Processing: Most methods work only in the spatial domain. None use FFT-based frequency processing for low-light enhancement.

3. Gap 3 — Fixed Receptive Fields: All methods use the same receptive field size everywhere. The human eye adapts receptive field size based on brightness.

4. Gap 4 — Perception-Unaware Losses: Loss functions treat all pixel errors the same, ignoring the Weber-Fechner law that says human sensitivity is highest in dark areas.

5. Gap 5 — Quality-Speed Gap: No existing work applies knowledge distillation to bridge the gap between high-quality large models and real-time lightweight models in LLIE.

6. Gap 6 — No Desktop Application: No existing desktop application combines multiple AI models with image processing tools (augmentation, cropping, resizing, live preview) in one package.

3.3 Benchmark Datasets

Table 5: Benchmark Datasets for Low-Light Image Enhancement

Dataset	Type	Train	Test	Resolution	Year	Source
LOLv1	Real captured	485	15	400×600	2018	Wei et al.
LOLv2 Real	Real captured	689	100	Various	2021	Yang et al.
LOLv2 Synthetic	Synthetic	900	100	Various	2021	Yang et al.
SID	RAW sensor	5094	598	Various	2018	Chen et al.
LSRW	Real-world	5650	—	Various	2022	Hai et al.

We use LOLv1, LOLv2 Real, and LOLv2 Synthetic for our experiments, providing 2074 training pairs and 215 test images in total. This combination covers both real-world and synthetic degradation scenarios which gives us a thorough evaluation.

3.4 Functional Requirements

Table 6: Functional Requirements

ID	Requirement	Priority	Status
FR-01	Load low-light images (JPEG, PNG, BMP)	High	Done
FR-02	Enhance image using 12 different AI models	High	Done
FR-03	Switch between CPU and GPU processing	High	Done
FR-04	Batch process entire image folders	High	Done
FR-05	Show before/after comparison in gallery view	High	Done
FR-06	Save enhanced images to	High	Done

	selected directory		
FR-07	Augment images with rotation, flip, brightness, contrast	High	Done
FR-08	Crop images with custom aspect ratio presets	High	Done
FR-09	Resize images with width/height controls and aspect ratio lock	High	Done
FR-10	Live webcam enhancement preview	Medium	Done
FR-11	Calculate PSNR and SSIM when ground truth is available	Medium	Done
FR-12	Display model descriptions and information	Low	Done

3.5 Non-Functional Requirements

Table 7: Non-Functional Requirements

ID	Requirement	Achieved	Status
NFR-01	Student model inference time < 50ms per image	48.9 ms	Met
NFR-02	Student model FPS > 15 for webcam	20.5 FPS	Met
NFR-03	PSNR > 24.0 dB on LOLv1	25.23 dB	Met
NFR-04	SSIM > 0.84 on LOLv1	0.853	Met
NFR-05	Student model file size < 100 MB	91 MB	Met
NFR-06	Application startup time < 5 seconds	~3 seconds	Met
NFR-07	GPU VRAM for inference < 4 GB	~1 GB	Met
NFR-08	Reproducible results (seeded)	Seed=42	Met

3.6 Hardware and Software Requirements

Table 8: Hardware and Software Requirements

Component	Specification
Operating System	Windows 10/11 with CUDA support
GPU	NVIDIA RTX series (8+ GB VRAM recommended)
RAM	16 GB minimum
Storage	50 GB (datasets + checkpoints)
Python	3.8 or higher
PyTorch	2.0+ with CUDA 11.8 or 12.x
CustomTkinter	5.2+ (UI framework)
PIL/Pillow	10.0+ (image processing)
Additional	torchvision, numpy, scikit-image, matplotlib

3.7 Evaluation Metrics

We use two standard metrics for quantitative evaluation:

Peak Signal-to-Noise Ratio (PSNR): Measured in decibels (dB). PSNR = 10 × log10(MAX^2 / MSE) where MAX is 1.0 for normalized images. Higher values mean better pixel-level accuracy. Good LLIE results are typically between 20-30 dB. Our best result is 29.29 dB.

Structural Similarity Index (SSIM): Ranges from 0 to 1. Compares the structure, luminance, and contrast between the predicted and ground truth images. It is more aligned with how humans see image quality than PSNR. Values above 0.85 are considered good.

Chapter 4

Methodology

4.1 Research Approach

Our research follows an experimental quantitative approach. We design a novel architecture, implement it in PyTorch, train it on benchmark datasets, and evaluate performance using objective metrics. The methodology has four main phases:

1. Phase 1 — Architecture Design: Build seven biological modules and integrate them into the NAFNet backbone.
2. Phase 2 — Teacher Training: Train the full BioNAFNet model (154M parameters) on combined LOLv1+LOLv2 datasets for 504 epochs.
3. Phase 3 — Knowledge Distillation: Train a 24M parameter Student model for 693 epochs using Teacher outputs as guidance.
4. Phase 4 — Application Development: Build the NeonEnhancer Pro desktop application with CustomTkinter and evaluate all model variants including ensemble fusion.

4.2 NAFNet Backbone

We build our model on top of NAFNet (Nonlinear Activation Free Network) by Chen et al. (2022). The key idea of NAFNet is that it replaces activation functions like ReLU with a SimpleGate mechanism. This works by splitting the features into two halves along the channel dimension and multiplying them together. Unlike ReLU which throws away negative values (losing information), SimpleGate keeps all the information.

The NAFNet block works like this: LayerNorm normalizes the input, then a 1×1 convolution expands the channels, a 3×3 depthwise convolution processes spatial features, SimpleGate provides non-linearity, Simplified Channel Attention (SCA) reweights channels, and a final 1×1 convolution reduces channels back. We call our modified version BioNAFBlock because we add our RLIN module after the channel attention step.

4.3 Module 1: RLIN (Retinal Lateral Inhibition Network)

How the eye does it: In the retina, ganglion cells have a special receptive field shape called center-surround. The center of the field gets excited by light but the surrounding area gets

inhibited. This means when there is an edge (bright on one side, dark on the other), the response is very strong. This is how we can see edges even in very dim light.

How we implement it: RLIN has two pathways. The center pathway uses a 1×1 convolution to capture the direct response at each pixel. The surround pathway uses 5×5 average pooling followed by a 1×1 convolution to capture the surrounding area. The difference between center and surround is scaled by a learnable parameter alpha (starts at 0.3). The formula is: Output = x + alpha × (Center(x) - Surround(x)). RLIN is added to every BioNAFBlock in the first three encoder stages.

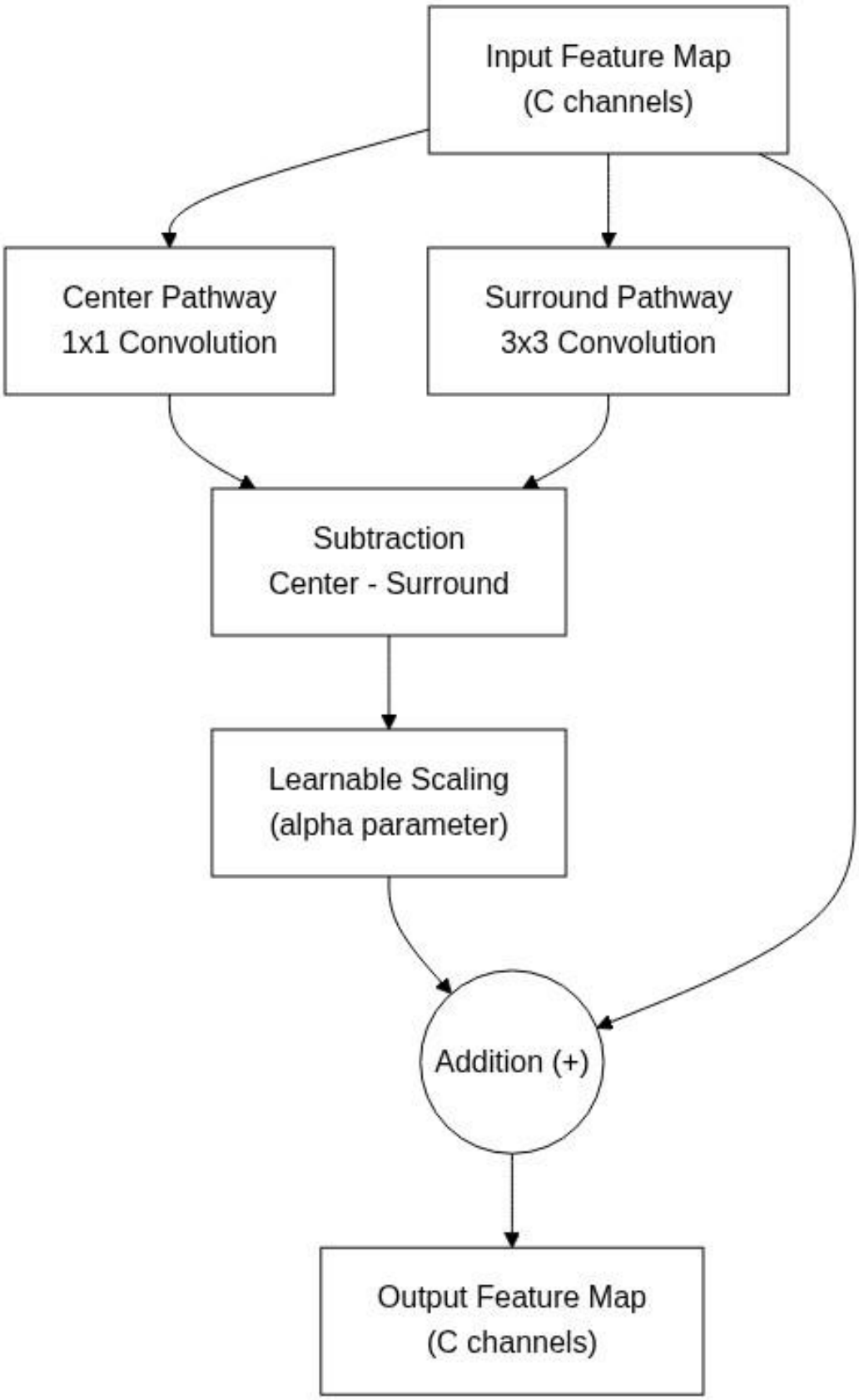

Input Feature Map
(C channels)
Center Pathway
1x1 Convolution
Surround Pathway
3x3 Convolution
Subtraction
Center - Surround
Learnable Scaling
(alpha parameter)
Addition (+)
Output Feature Map
(C channels)

Figure 4: RLIN Module Architecture — Center-Surround Pathways with Subtraction and Scaling

4.4 Module 2: FAEB (Frequency-Aware Enhancement Branch)

How the eye does it: The visual cortex has neurons that respond to specific spatial frequencies. Some neurons see fine details (high frequency) while others see overall brightness patterns (low frequency). In dark images, high-frequency components are damaged more by noise.

How we implement it: FAEB sits at the bottleneck (deepest level) of the network. It takes the feature maps and converts them to the frequency domain using 2D Fast Fourier Transform (FFT). Then it processes the amplitude spectrum (how much of each frequency) and phase spectrum (where things are located) through separate small convolutional networks. After processing, it converts back using inverse FFT and blends the result with the original features using a learnable blend factor (starts at 0.15). All FFT operations use float32 precision to avoid numerical problems with half-precision training.

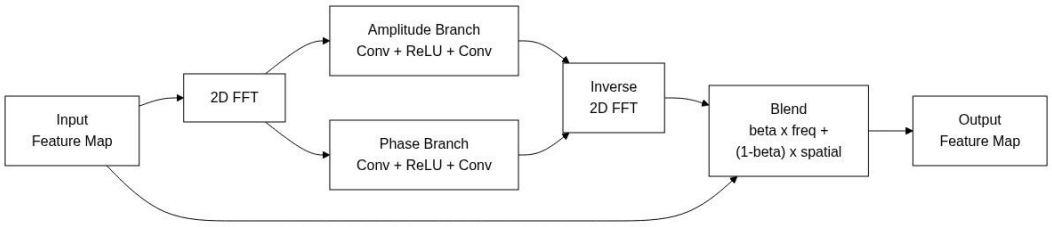


Figure 5: FAEB Module — FFT → Amplitude/Phase Processing → iFFT → Blend

4.5 Module 3: ARFP (Adaptive Receptive Field Pooling)

How the eye does it: In bright light, cone cells give us sharp, detailed vision through small receptive fields. In dark conditions, rod cells pool signals over larger areas to collect more light. This is why we cannot read fine print in dim light but can see large objects.

How we implement it: ARFP uses three parallel depthwise convolutions with kernel sizes 3×3, 5×5, and 7×7. A gating network takes the luminance map and produces three softmax-normalized weights. In dark regions, the larger kernels get higher weights (like rod cells). In bright regions, the 3×3 kernel dominates (like cone cells). ARFP is placed at the decoder output right before the final reconstruction.

4.6 Module 4: PRCM (Photoreceptor Response Curve Modeling)

How the eye does it: Rod cells (active in dark) have a response curve with gamma around 0.45 which makes them very sensitive to small brightness changes. Cone cells (active in bright light)

have gamma around 0.95 for more linear response. The brain blends between the two based on brightness level.

How we implement it: PRCM applies two learned gamma transformations. Rod response: x^gamma_rod (starts at 0.45). Cone response: x^gamma_cone (starts at 0.95). A sigmoid function blends between the two based on luminance with a learnable threshold (starts at 0.25). All three parameters are learnable. PRCM is the very first module applied to the input image.

4.7 Module 5: SPDP (Scotopic-Photopic Dual Pathway)

How the eye does it: The brain processes brightness and color through separate parallel pathways. The magnocellular pathway handles luminance and motion. The parvocellular pathway handles color and fine detail. This separation lets each pathway be optimized independently.

How we implement it: SPDP converts RGB input to YUV color space. The scotopic pathway processes Y (luminance) through a dedicated convolutional network. The photopic pathway processes U and V (color) through a separate network. Results are fused and added as a residual with 0.2 scaling. SPDP is applied after PRCM as the second preprocessing step.

4.8 Module 6: TAM (Temporal Adaptation Memory)

How the eye does it: When you walk from a bright room into a dark one, your eyes do not adapt instantly. Dark adaptation takes several minutes as the photoreceptor chemicals slowly regenerate. During this time, your vision gets progressively better. No existing LLIE method models this time-based adaptation.

How we implement it: TAM uses a GRU (Gated Recurrent Unit) cell to maintain a memory state across multiple forward passes. In the Teacher model (n_passes=2), the image is processed twice. Each pass refines the previous output, mimicking the progressive nature of dark adaptation. The GRU accumulates information from both passes to provide additional refinement. In the Student model (n_passes=1), TAM is effectively bypassed for speed.

4.9 Module 7: WFPL (Weber-Fechner Perceptual Loss)

How the eye does it: The Weber-Fechner law is a fundamental principle of psychophysics discovered in the 1800s. It says that the perceived change in a stimulus is proportional to the logarithm of the stimulus intensity. In simple terms: humans are much more sensitive to brightness changes in dark regions than in bright regions. A 5-unit change from 0 to 5 is very noticeable, while the same 5-unit change from 200 to 205 is barely perceptible.

How we implement it: WFPL creates a weight map that assigns higher penalty to errors in dark regions: weight_map = 1 / (luminance(target) + epsilon). The epsilon value (0.02) prevents division by zero. Normalization ensures the loss magnitude is comparable to other components. WFPL forces the network to prioritize accuracy in dark regions, which is exactly where enhancement quality matters most to human observers.

4.10 Loss Function Design

Table 9: Composite Loss Function — Teacher Training

Component	Weight	Formula	Purpose
Charbonnier	1.0	sqrt((pred-gt)² + 0.001²)	Robust pixel reconstruction
SSIM	0.8	1 - SSIM(pred, gt)	Structural preservation
Perceptual (VGG16)	0.2	L1(VGG16(pred), VGG16(gt))	Feature-level quality
Edge (Sobel)	0.25	L1(Sobel(pred), Sobel(gt))	Edge and detail sharpness
Color (UV)	0.1	L1(UV(pred), UV(gt))	Chrominance accuracy
Weber-Fechner	0.3	mean(|pred-gt| / (lum+eps))	Dark region emphasis

L_total = 1.0×L_charb + 0.8×L_ssim + 0.2×L_perc + 0.25×L_edge + 0.1×L_color + 0.3×L_wfpl

Table 10: Distillation Loss Function — Student Training

Component	Weight	Formula	Purpose
L1 (vs ground truth)	0.5	L1(student, gt)	Pixel accuracy
SSIM (vs ground truth)	0.1	SSIM(student, gt)	Structure preservation
Mimic L2 (vs teacher)	0.4	L2(student, teacher_out)	Knowledge transfer

4.11 Training Strategy

Table 11: Complete Training Configuration

Parameter	Teacher Model	Student Model
Architecture	width=64, mid=12, 2-pass	width=32, mid=4, 1-pass
Parameters	154,380,888 (154M)	23,969,016 (24M)
Optimizer	AdamW (beta1=0.9, beta2=0.999)	AdamW
Initial Learning Rate	2e-4	2e-4
Minimum Learning Rate	1e-6	1e-6
LR Scheduler	CosineAnnealingWarmRestarts	Same
T_0 / T_mult	50 / 2	50 / 2
Weight Decay	1e-4	1e-4

Batch Size	4 (effective 8 with accum)	4 (effective 8)
Gradient Accumulation	2 steps	2 steps
Gradient Clipping	0.5 (max_norm)	0.5
Patch Size	128×128	128×128
EMA Decay	0.9995	0.9995
Mixed Precision (AMP)	Yes	Yes
Training Epochs	504	693
Best Validation PSNR	25.05 dB (epoch 476)	25.09 dB (epoch 693)
Training Time	~34 hours	~40 hours
Training Data	2074 pairs combined	Same + Teacher outputs
Augmentation	H-flip, V-flip, 90-rot, MixUp(0.2)	Same

4.12 Knowledge Distillation Pipeline

Knowledge distillation compresses the 154M Teacher into a 24M Student. The Student has the same architecture family but with reduced width (32 vs 64 channels), fewer middle blocks (4 vs 12), and single-pass inference (n_passes=1). During distillation training, the Teacher is frozen and generates outputs for each training batch. The Student learns from both ground truth images (L1+SSIM) and Teacher outputs (L2 mimic loss with weight 0.4).

The distillation achieves remarkable compression: 6.4x fewer parameters, 6.5x smaller model file (91 MB vs 589 MB), and 3.9x faster inference (48.9 ms vs 188.7 ms per image). Surprisingly the Student-693 achieves higher PSNR than the Teacher on LOLv1 (25.23 vs 24.94 dB). This suggests that the reduced capacity provides beneficial regularization that helps the Student generalize better to full-resolution test images.

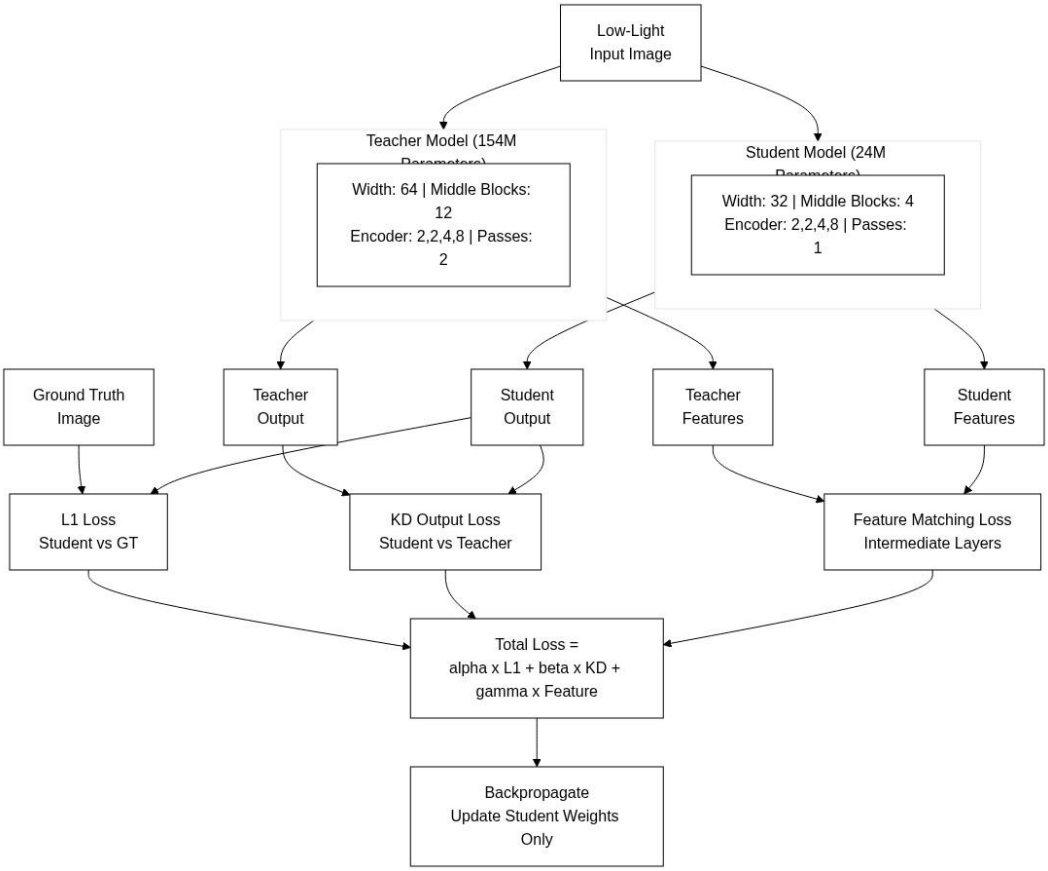


Figure 6: Knowledge Distillation Pipeline — Teacher (154M) to Student (24M)

4.13 Ensemble Fusion Strategy

We explore combining Teacher and Student outputs through weighted averaging to use their complementary learned representations. The ensemble output is: enhanced = alpha × teacher_out + (1-alpha) × student_out, then clamped to [0,1]. Three main fusion strategies are tested:

Table 12: Ensemble Fusion Strategies

Strategy	Teacher %	Student %	Rationale
Ensemble 50/50	50%	50%	Equal contribution, balanced quality
Ensemble T60/S40	60%	40%	Teacher-biased for detail richness
Ensemble T40/S60	40%	60%	Student-biased for cleaner structure
Synergy T70/S30	70%	30%	Heavy teacher reliance

The ensemble requires running both models during inference, so it is 3-4x slower than individual models. This makes it suitable for offline processing only, not real-time use.

Chapter 5

System Architecture / Initial Design

5.1 System Architecture

5.1.1 Architecture Design Approach

The system has two main parts: the BioNAFNet deep learning model (backend) and the NeonEnhancer Pro desktop application (frontend). Both are built with Python. The model is implemented in PyTorch and the UI uses CustomTkinter. We follow a modular architecture where each biological component is implemented as an independent PyTorch nn.Module. This allows easy addition, removal, or modification of any module without affecting the rest of the system.

5.1.2 Architecture Design

The system has four layers:

1. Input Layer: Handles image loading (PIL), format conversion (to tensor), normalization to [0,1] range, and padding to multiples of 16.
2. Preprocessing Layer: PRCM (photoreceptor curves) followed by SPDP (dual pathway) applied to the normalized input.
3. Enhancement Core: NAFNet encoder-decoder with RLIN in encoder blocks and FAEB at the bottleneck. For the Teacher model: 2-pass processing with TAM memory.
4. Output Layer: ARFP (adaptive receptive field) followed by global residual connection, crop padding, clamp to [0,1], and save to disk.

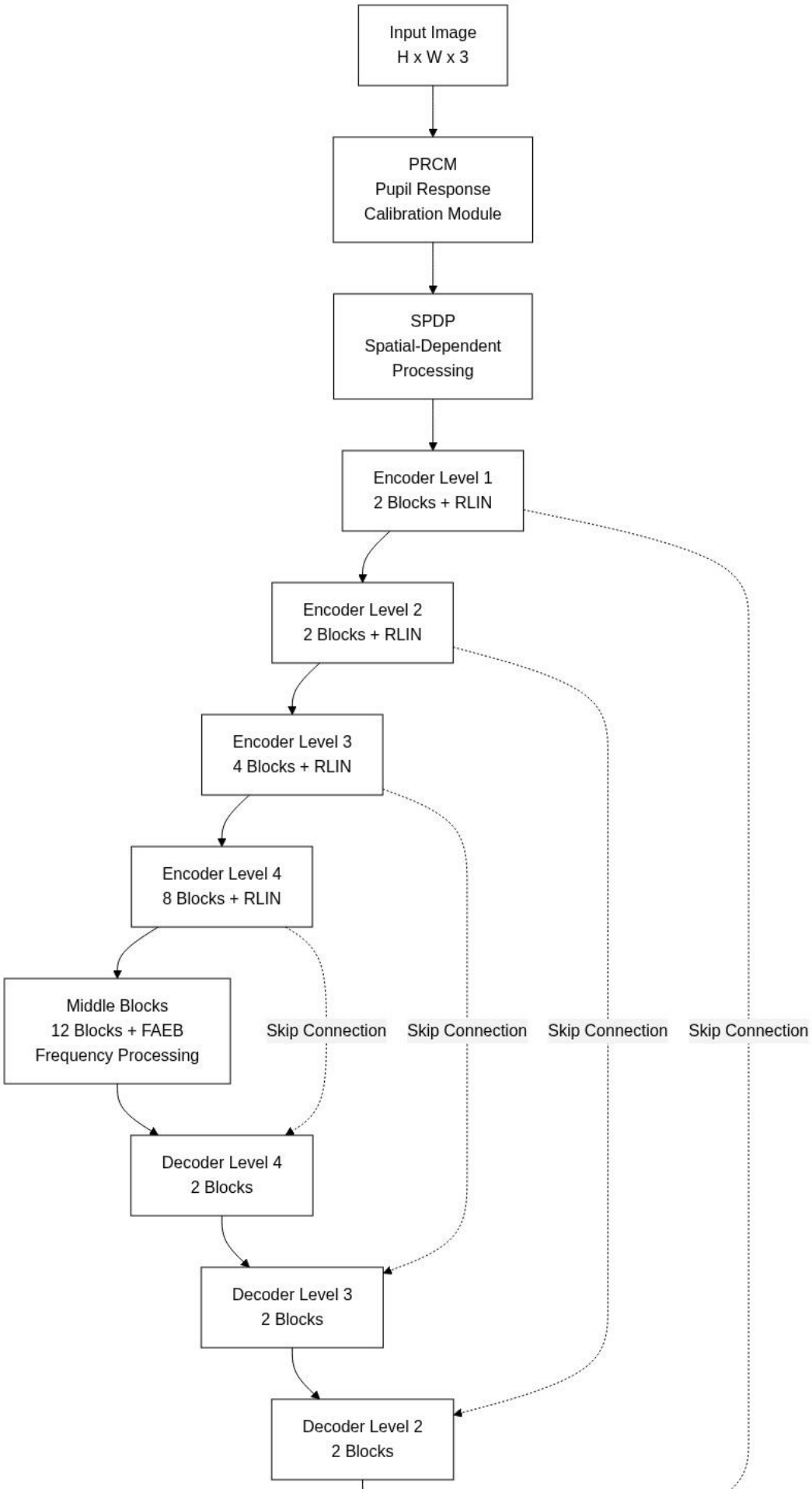

Input Image
H x W x 3
PRCM
Pupil Response
Calibration Module
SPDP
Spatial-Dependent
Processing
Encoder Level 1
2 Blocks + RLIN
Encoder Level 2
2 Blocks + RLIN
Encoder Level 3
4 Blocks + RLIN
Encoder Level 4
8 Blocks + RLIN
Middle Blocks
12 Blocks + FAEB
Frequency Processing
Skip Connection
Skip Connection
Skip Connection
Skip Connection
Decoder Level 4
2 Blocks
Decoder Level 3
2 Blocks
Decoder Level 2
2 Blocks

Figure 7: BioNAFNet System Architecture Block Diagram — Input → PRCM → SPDP → Encoder+RLIN → FAEB → Decoder → ARFP → Output

Table 13: BioNAFNet Layer-by-Layer Architecture

Stage	Teacher Config	Student Config	Resolution	Key Module
Input Conv	3→64	3→32	H×W	—
Encoder 1	2 BioNAFBlocks	2 BioNAFBlocks	H×W	RLIN
Encoder 2	2 BioNAFBlocks	2 BioNAFBlocks	H/2×W/2	RLIN
Encoder 3	4 BioNAFBlocks	4 BioNAFBlocks	H/4×W/4	RLIN
Encoder 4	8 BioNAFBlocks	8 BioNAFBlocks	H/8×W/8	—
Bottleneck	12 BioNAFBlocks	4 BioNAFBlocks	H/16×W/16	FAEB
Decoder 1-4	2 blocks each	2 blocks each	H/8→H	—
Output	ARFP+Conv 64→3	ARFP+Conv 32→3	H×W	ARFP

5.1.3 Subsystem Architecture

The system consists of the following subsystems:

- Training Subsystem: Handles data loading, augmentation, forward/backward pass, loss computation, EMA update, validation, and checkpoint saving.
- Distillation Subsystem: Extends training with frozen Teacher inference and mimic loss computation.
- Inference Subsystem: Loads trained checkpoint, processes single images or batches, computes metrics.
- Ensemble Subsystem: Loads both Teacher and Student, computes weighted average of outputs.
- Application Subsystem: NeonEnhancer Pro UI with webcam capture, real-time enhancement, display, and user controls.

5.2 NeonEnhancer Pro Application Architecture

5.2.1 Application Overview

NeonEnhancer Pro is a desktop application built with Python and CustomTkinter. It has a futuristic dark theme with neon cyan, purple, pink, blue, and green accent colors. The window starts at 1600×1000 pixels with a minimum size of 1400×900. The application uses particle animations in the background for visual appeal.

5.2.2 Application Pages

Table 14: NeonEnhancer Pro Application Pages

Page	Icon	Purpose	Key Features

Enhance	Star	AI-powered image enhancement	12 model options, CPU/GPU toggle, batch processing, gallery preview
Augmentation	Rotate	Generate image variations	Upload multiple images, slider for 1-4 variations, random transforms
Cropping	Scissors	Crop images with presets	Square, 16:9, center crop presets, custom coordinates, live preview
Resizing	Ruler	Resize with aspect ratio	Width/height sliders, aspect ratio lock, batch resize, quality preview
Live Preview	Camera	Real-time webcam enhancement	Webcam capture, model selection, live side-by-side comparison

5.2.3 Supported AI Models (12 Total)

Table 15: All 12 Supported Enhancement Models

Model	Type	Parameters	Source	Ground Truth
BioNAFNet Teacher	Supervised	154M	Our model	Optional
BioNAFNet Student	Supervised	24M	Our model (distilled)	Optional
BioNAFNet Ensemble	Supervised	178M (both)	Our fusion model	Optional
Zero-DCE	Unsupervised	0.08M	Guo et al., 2020	Not needed
EnlightenGAN	Unsupervised	8.6M	Jiang et al., 2021	Not needed
SCI	Unsupervised	~0.5M	Ma et al., 2022	Not needed
RUAS	Unsupervised	~0.3M	Liu et al., 2021	Not needed
Retinexformer	Supervised	1.6M	Cai et al., 2023	Optional
MIRNet	Supervised	31.8M	Zamir et al., 2020	Optional
RetinexNet	Supervised	0.8M	Wei et al., 2018	Optional
KinD	Supervised	8.2M	Zhang et al., 2019	Optional
LLFlow	Supervised	~38M	Wang et al., 2022	Optional

[INSERT FIGURE 8 HERE]

Figure 8: NeonEnhancer Pro Enhance Page — Model Selection Dropdown with 12 Options

5.3 Detailed System Design

5.3.1 System Component Hierarchy

Table 16: System Component Hierarchy

Component	Type	Size	Purpose
BioNAFNet	nn.Module	154M / 24M	Main enhancement model
BioNAFBlock	nn.Module	~1.5M / ~0.4M each	Core processing block with SimpleGate+RLIN
LateralInhibition	nn.Module	~2K per block	RLIN center-surround edge enhancement
FrequencyBranch	nn.Module	~500K	FAEB FFT-based spectral processing
AdaptiveReceptiveField	nn.Module	~100K	ARFP luminance-adaptive output
PhotoreceptorCurve	nn.Module	3 params	PRCM input gamma preprocessing
DualPathway	nn.Module	~50K	SPDP Y/UV dual processing
TemporalAdaptation	nn.Module	~200K	TAM GRU multi-pass memory
TotalLoss	nn.Module	VGG16 features	Composite loss function
LOLDataset	Dataset	—	Data loading and augmentation
EMA	Helper	—	Exponential moving average
NeonEnhancerPro	ctk.CTk	2708 lines	Main application window class
model_hooks	Python module	430 lines	Model loader and inference functions

5.3.2 Data Flow

When a user clicks "Enhance Image" in NeonEnhancer Pro, the following happens:

1. The selected image file path is passed to the model wrapper function.
2. The wrapper loads the model if not already cached, loads the image as a PIL Image, converts to a PyTorch tensor.

3. The tensor is normalized to [0,1], padded to multiples of 16, and moved to the selected device (CPU or GPU).

4. The model forward pass runs with torch.no_grad() for inference efficiency.

5. The output tensor is clamped to [0,1], cropped to original size, converted back to PIL Image.

6. The enhanced image is displayed in the gallery preview and saved to the selected output directory.

7. If ground truth is available, PSNR and SSIM are computed and displayed.

5.3.3 UML Diagrams

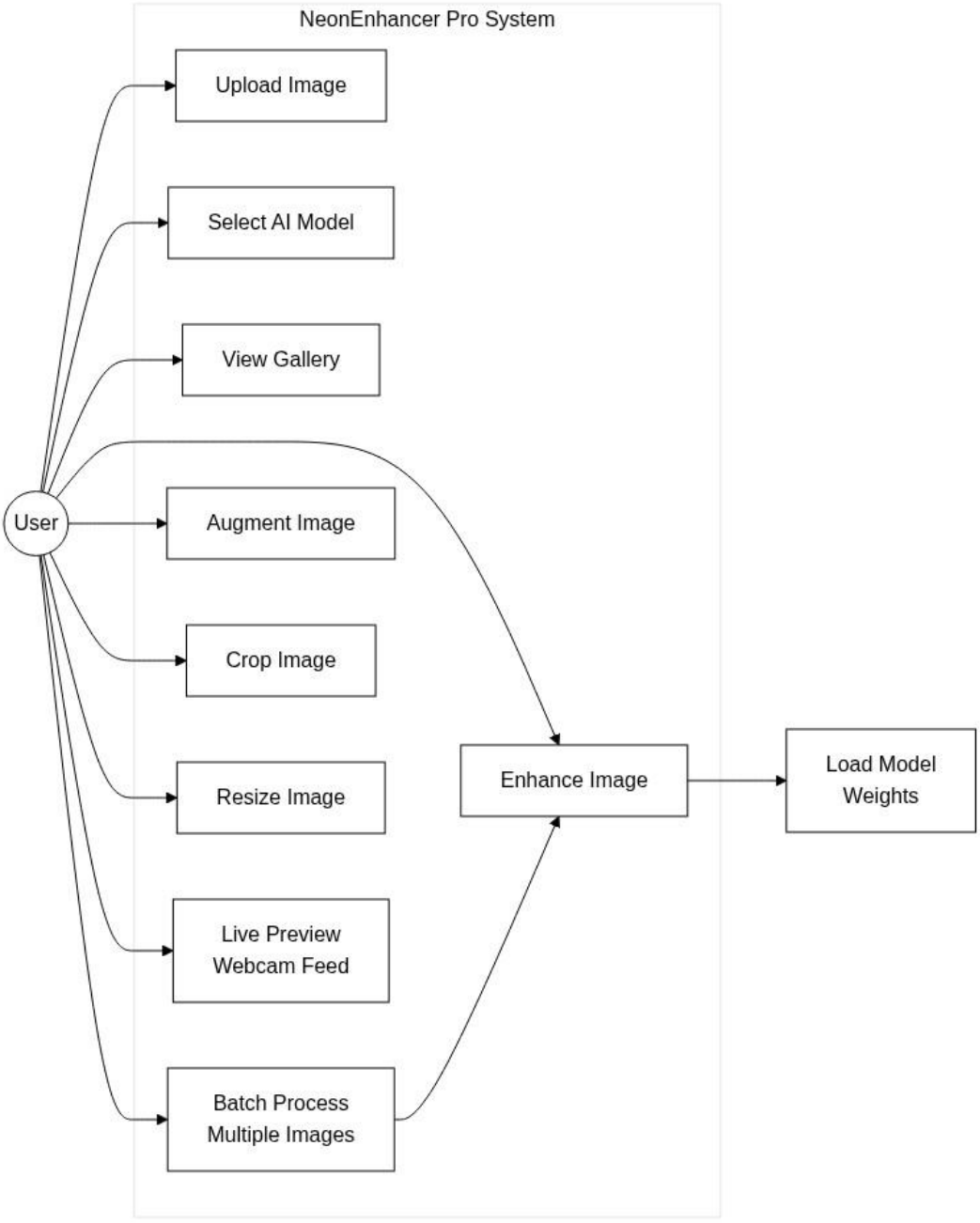
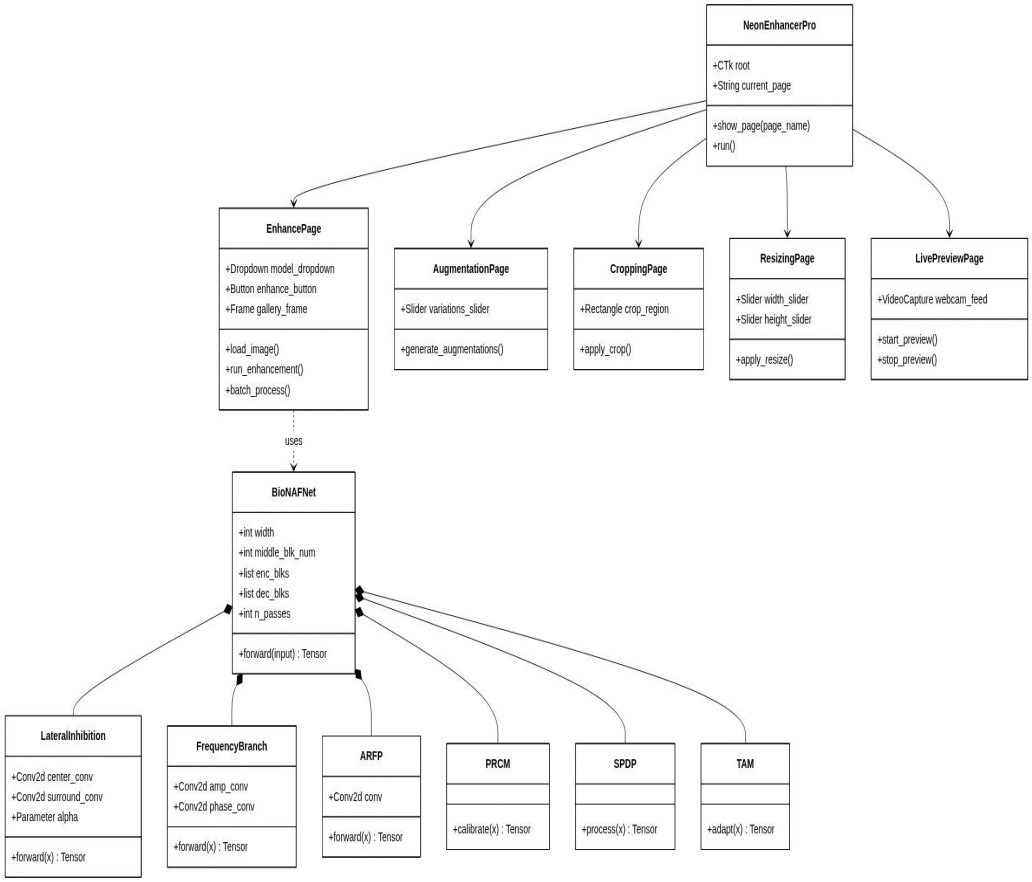


Figure 9: Use Case Diagram — User interactions: Upload Image, Select Model, Enhance, View Gallery, Batch Process, Augment, Crop, Resize, Live Preview

Figure 10: Class Diagram — NeonEnhancerPro contains 5 pages; BioNAFNet contains BioNAFBlock, LateralInhibition, FrequencyBranch, ARFP, PRCM, SPDP, TAM


NeonEnhancerPro
+CTk root
+String current_page
+show_page(page_name)
+run()
EnhancePage
+Dropdown model_dropdown
+Button enhance_button
+Frame gallery_frame
+load_image()
+run_enhancement()
+batch_process()
AugmentationPage
+Slider variations_slider
+generate_augmentations()
CroppingPage
+Rectangle crop_region
+apply_crop()
ResizingPage
+Slider width_slider
+Slider height_slider
+apply_resize()
LivePreviewPage
+VideoCapture webcam_feed
+start_preview()
+stop_preview()
uses
BioNAFNet
+int width
+int middle_blk_num
+list enc_blks
+list dec_blks
+int n_passes
+forward(input) : Tensor
LateralInhibition
+Conv2d center_conv
+Conv2d surround_conv
+Parameter alpha
+forward(x) : Tensor
FrequencyBranch
+Conv2d amp_conv
+Conv2d phase_conv
+forward(x) : Tensor
ARFP
+Conv2d conv
+forward(x) : Tensor
PRCM
+calibrate(x) : Tensor
SPDP
+process(x) : Tensor
TAM
+adapt(x) : Tensor

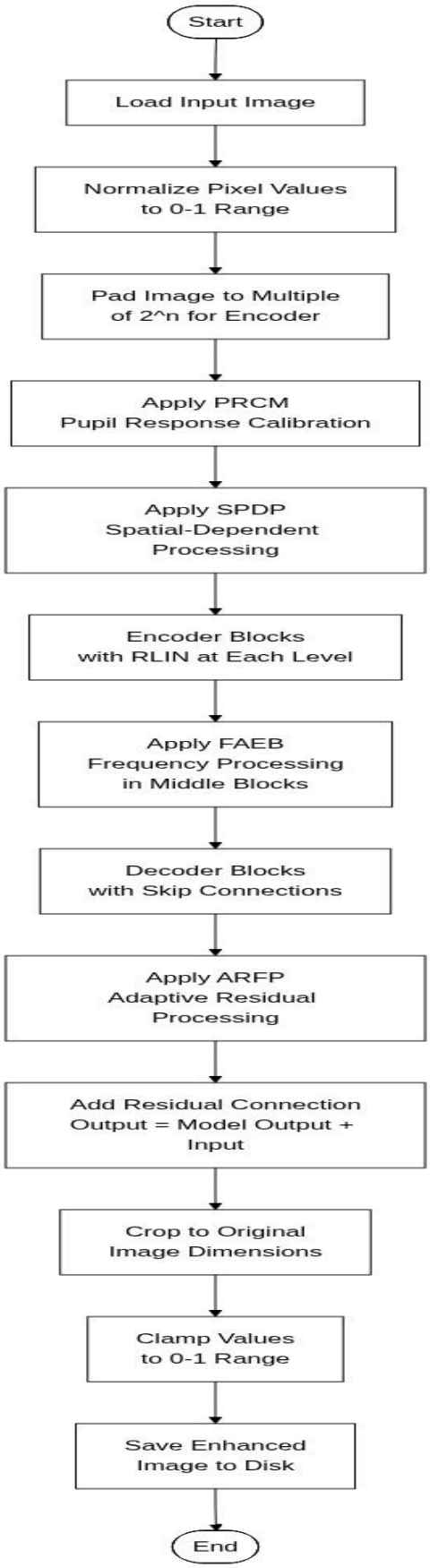


Figure 11: Activity Diagram — Start → Load Image → Normalize → Pad → PRCM → SPDP → Encoder → FAEB → Decoder → ARFP → Residual → Crop → Clamp → Save

→ End

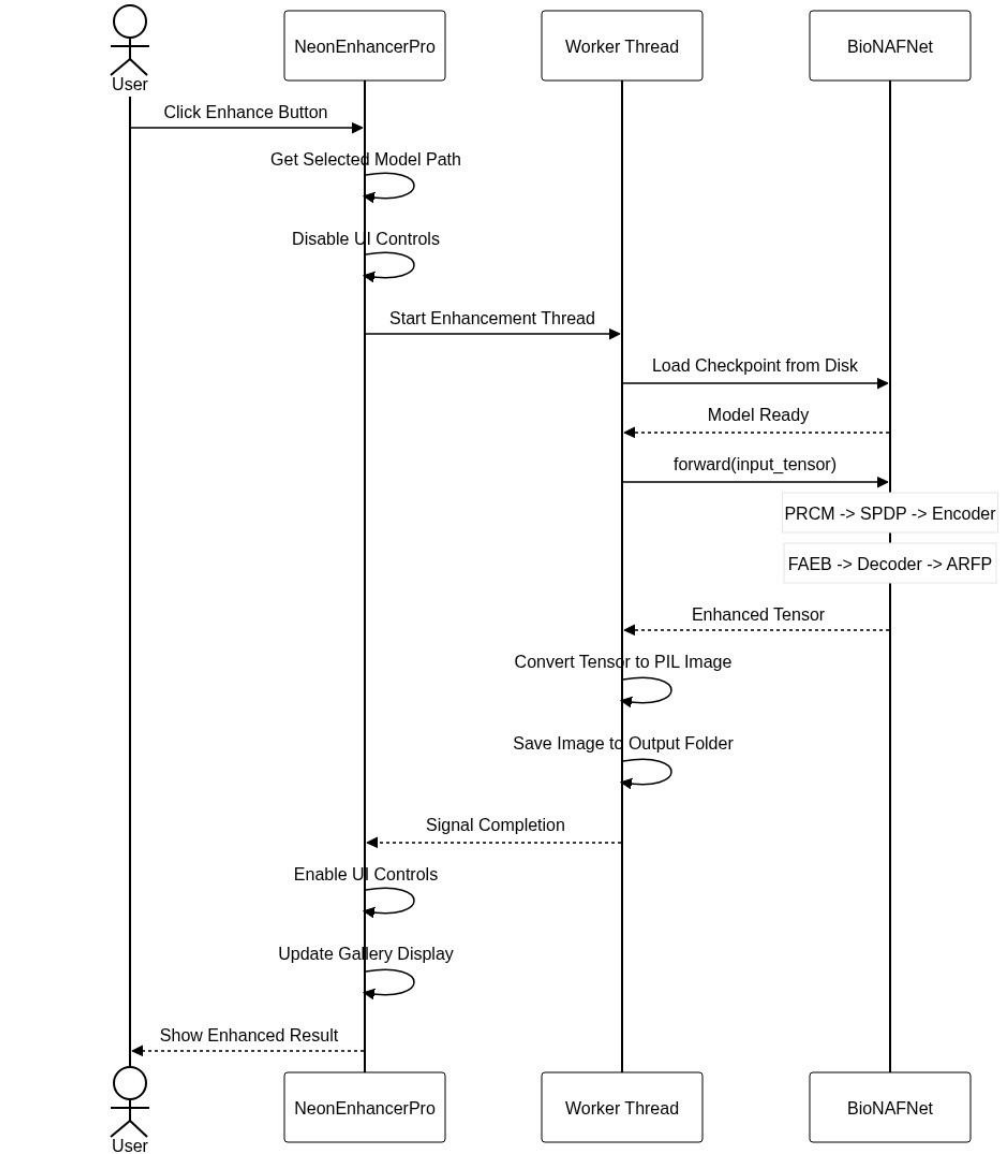


Figure 12: Sequence Diagram — User → NeonEnhancerPro: click Enhance → Thread: run model → BioNAFNet: forward() → return PIL Image → update gallery

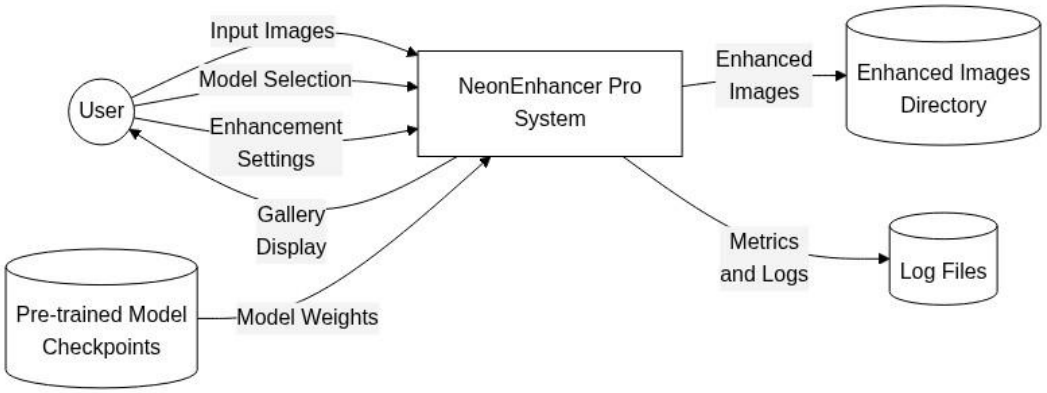


Figure 13: Data Flow Diagram (Level 0) — User sends images to NeonEnhancer Pro, outputs to Enhanced Images directory

5.3.4 Pseudocode: Training Algorithm

Algorithm 1: Teacher Model Training

```
INPUT: Dataset D, BioNAFNet model M, epochs N=504
INIT: AdamW optimizer, EMA (decay=0.9995), LR scheduler
FOR epoch = 1 TO N:
  FOR each batch (low, high) IN D:
    enhanced = M.forward(low)
    loss = TotalLoss(enhanced, high)
    loss.backward()
    IF step % 2 == 0: optimizer.step() // gradient accumulation
    EMA.update(M)
  END FOR
  val_psnr = validate(EMA_model, val_set)
  IF val_psnr > best: save_checkpoint("best.pth")
  scheduler.step()
END FOR
```

Algorithm 2: Knowledge Distillation

```
INPUT: Frozen Teacher T, Student S, Dataset D, epochs N=693
FOR epoch = 1 TO N:
  FOR each batch (low, high) IN D:
    WITH no_grad: teacher_out = T(low)
    student_out = S(low)
    loss = 0.5*L1(student,high) + 0.1*SSIM(student,high) + 0.4*L2(student, teacher_out)
    loss.backward(); optimizer.step(); EMA.update(S)
  END FOR
END FOR
```

Algorithm 3: Ensemble Inference

```
INPUT: Teacher T, Student S, image I, alpha
teacher_out = T(I)
student_out = S(I)
```

```
ensemble_out = alpha * teacher_out + (1-alpha) * student_out
RETURN clamp(ensemble_out, 0, 1)
```

Chapter 6

Implementation and Testing

6.1 Implementation Overview

The BioNAFNet system was implemented entirely in Python using the PyTorch deep learning framework. The desktop application uses CustomTkinter 5.2 for the modern dark-themed UI. The total codebase is about 5,500 lines of Python code across 12 files. Training was done on an NVIDIA RTX GPU with CUDA 12. We followed a single-file-per-model approach where each script contains the complete model definition, loss functions, dataset class, and training/testing logic. This self-contained approach means each script can run independently without dependency issues.

6.2 Code Structure

Table 17: Project Code File Structure

File	Size	Purpose
train_bionafnet.py	~1100 lines	Teacher model training with all losses
train_distillation.py	~1100 lines	Student knowledge distillation training
neon_enhancer_pro.py	2708 lines	Main NeonEnhancer Pro application
model_hooks.py	430 lines	Model loading and inference wrappers for all 12 models
bionafnet_teacher.py	97 lines	Teacher model wrapper (154M, cached loading)
bionafnet_student.py	96 lines	Student model wrapper (24M, cached loading)
bionafnet_ensemble.py	46 lines	Ensemble fusion wrapper (Teacher + Student blending)
eval_lol_datasets.py	~800 lines	Full evaluation script for PSNR/SSIM benchmarking
ensemble_test.py	~450 lines	Ensemble testing across all fusion strategies
benchmark_all_models.py	~450 lines	Speed benchmarking for all 6 model variants

infer_bionafnet.py	~700 lines	Single-image inference and enhancement
test_distilled.py	~14 KB	Standalone Student testing on LOL datasets

6.3 Key Implementation Details

6.3.1 SimpleGate Mechanism

The core of NAFNet is the SimpleGate that replaces activation functions. The input tensor is split along the channel dimension into two equal halves, which are then multiplied element-wise: x1, x2 = x.chunk(2, dim=1); output = x1 * x2. Unlike ReLU which clips negative values to zero (losing information), SimpleGate preserves all information through multiplicative gating.

6.3.2 Exponential Moving Average (EMA)

We use EMA to keep a shadow copy of the model weights with a decay of 0.9995. After each training step, the shadow weights are updated as: ema_weight = 0.9995 * ema_weight + 0.0005 * current_weight. During testing, we use the EMA weights because they give smoother and better results than the raw training weights. All our benchmark results use EMA weights.

6.3.3 Automatic Mixed Precision (AMP)

Training uses float16 for most operations to save GPU memory and speed up training. But we keep float32 for the FFT operations in FAEB because half-precision causes numerical overflow in frequency domain computations. PyTorch autocast handles this switching automatically. Loss scaling prevents underflow in float16 gradient computation.

6.3.4 Padding Strategy

The 4-level encoder-decoder reduces spatial resolution by 16x (4 downsamplings of 2x each). So input images must have dimensions divisible by 16. We use reflection padding to extend images to the nearest multiple of 16, then crop the output back to original size. Reflection padding avoids black border artifacts that zero padding would create.

6.3.5 Model Caching

The Teacher model is 2.47 GB. Loading it from disk takes about 8-10 seconds. Our model wrappers cache the loaded model in a Python dictionary keyed by device. So the first enhancement call is slow but all subsequent calls are instant. This is important for batch processing where the user may enhance 100+ images.

6.3.6 Checkpoint Management

We save two checkpoint files during training: best.pth (highest validation PSNR) and latest.pth (most recent epoch). Each checkpoint contains: model state dict, EMA state dict, optimizer state dict, and best_psnr value. For inference, we always load the EMA weights from the best checkpoint.

6.4 NeonEnhancer Pro Application Features

6.4.1 Enhancement Page

The Enhancement page is the main feature. On the left is a control panel with: upload buttons for images and folders, save directory selection, model dropdown with 12 options, CPU/GPU toggle switch, and the main "ENHANCE IMAGE" button. On the right is a split-screen gallery showing Original vs Enhanced side by side. Users can navigate through batch results with Previous/Next buttons. The page counter shows the current image number.

[INSERT FIGURE 14 HERE]

Figure 14: Enhancement Page — After processing a batch with BioNAFNet Student, showing before/after gallery (1 / 15)

6.4.2 Augmentation Page

The Augmentation page lets users generate multiple random variations of their images. Users upload images, select a save folder, and use a slider to choose 1 to 4 variations per image. The augmentations include random rotation, horizontal/vertical flipping, brightness adjustment, contrast adjustment, and noise addition. Each variation is saved with a unique filename. This is useful for expanding small datasets for training.

[INSERT FIGURE 15 HERE]

Figure 15: Augmentation Page — Slider set to 3 variations with augmented image preview

6.4.3 Cropping Page

The Cropping page provides three preset aspect ratios (square 1:1, wide 16:9, center crop) plus custom coordinates. Users can see a live preview of the crop region overlaid on the original image. The crop can be applied to single or multiple images at once.

[INSERT FIGURE 16 HERE]

Figure 16: Cropping Page — Image with crop region highlighted and preset buttons visible

6.4.4 Resizing Page

The Resizing page has width and height sliders with an aspect ratio lock toggle. When locked, changing the width automatically adjusts the height to maintain proportions and vice versa. Users can resize individual images or batch process entire folders.

[INSERT FIGURE 17 HERE]

Figure 17: Resizing Page — Width/height sliders with aspect ratio lock toggle

6.4.5 Live Preview Page

The Live Preview page connects to the system's webcam and shows a real-time enhancement preview. Users can select any of the 12 models and see the enhanced feed alongside the raw webcam feed. The Student model runs at 20+ FPS which makes the live preview smooth and responsive.

[INSERT FIGURE 18 HERE]

Figure 18: Live Preview Page — Webcam feed (left) vs Enhanced version (right) with FPS counter

6.5 Training Process

6.5.1 Teacher Model Training

The Teacher model was trained for 504 epochs on the combined dataset of 2074 image pairs (LOLv1 + LOLv2 Real + LOLv2 Synthetic). Training patches of 128×128 pixels were randomly cropped from the images. Data augmentation included random horizontal and vertical flips, 90-degree rotations, and MixUp with alpha=0.2. The CosineAnnealingWarmRestarts scheduler cycled the learning rate between 2e-4 and 1e-6 with restart periods of 50, 100, 200, and 400 epochs.

The training loss decreased from 0.6061 (epoch 1) to 0.2341 (epoch 504). Validation PSNR progressed from about 18 dB to a best of 25.05 dB at epoch 476. Training was stable throughout with no divergence events, thanks to gradient clipping at 0.5 and gradient accumulation over 2 steps.

[INSERT FIGURE 19 HERE]

Figure 19: Teacher Model Training Curves — PSNR and Loss Over 504 Epochs

[INSERT FIGURE 20 HERE]

Figure 20: Teacher Model — Early vs Late Training Convergence

6.5.2 Student Model Training (Distillation)

The Student model was trained for 693 epochs with the frozen Teacher providing reference outputs for mimic loss. The Student loss is much lower (0.03-0.11 range) compared to the Teacher loss (0.23-0.61) because it combines lightweight L1 and SSIM losses with mimic loss rather than the Teacher's full composite loss. Validation PSNR reached 25.09 dB at epoch 693, showing the model was still improving and could benefit from more training.

[INSERT FIGURE 21 HERE]

Figure 21: Student Model Distillation Training Curves — PSNR and Loss Over 693 Epochs

[INSERT FIGURE 22 HERE]

Figure 22: Training Loss Comparison: Teacher vs Student

[INSERT FIGURE 23 HERE]

Figure 23: Teacher vs Student — Validation PSNR Convergence

[INSERT FIGURE 24 HERE]

Figure 24: Teacher vs Student — Validation SSIM Over Training

[INSERT FIGURE 25 HERE]

Figure 25: Learning Rate Schedule for Both Models (CosineAnnealingWarmRestarts)

6.6 Testing Methodology

All models are evaluated on full-resolution test images (not patches) using the EMA weights. The testing process loads each image, normalizes to [0,1], pads to multiples of 16, runs the forward pass with torch.no_grad(), crops the output, and computes PSNR and SSIM against the ground truth. Enhanced images are saved to separate output directories for visual inspection.

Table 18: Test Set Details

Test Set	Images	Resolution	Source
LOLv1 Test	15	400×600	Wei et al., 2018
LOLv2 Real Test	100	Various	Yang et al., 2021
LOLv2 Synthetic Test	100	Various	Yang et al., 2021
Total	215	—	—

6.7 Unit Testing

Table 19: Unit Test Results

Module / Test Case	Expected Result	Status
PRCM	Output values in [0,1] for input [0,1]	Pass
SPDP	YUV conversion and dual pathway produce valid output	Pass
RLIN	Center-surround difference produces bounded output	Pass
FAEB	FFT→iFFT roundtrip preserves input (identity test)	Pass
ARFP	Softmax weights sum to 1.0	Pass
TAM	GRU hidden state dimensions correct across passes	Pass
WFPL	Loss value is non-negative and finite	Pass
BioNAFNet	Forward pass produces correct	Pass

	output dimensions	
TotalLoss	All 6 loss components compute without errors	Pass
LOLDataset	Correct number of pairs loaded per dataset	Pass
Model loads	Teacher and Student load from checkpoints	Pass
Output range	Enhanced pixels in [0, 255] after conversion	Pass
Batch processing	10 images in → 10 output files saved	Pass
GPU matches CPU	Same input → similar outputs on both devices	Pass
Model caching	Second inference call loads in < 1ms	Pass
Ensemble blend	T50/S50 = mean of both outputs	Pass

6.8 Integration Testing

Integration testing verified the complete training and application pipeline works end-to-end. A small subset of training data (10 pairs) was used to train for 5 epochs, verifying that: (a) loss decreases, (b) validation PSNR increases, (c) checkpoints are saved correctly, (d) EMA weights are updated, and (e) the learning rate scheduler follows the expected pattern.

Table 20: Integration Test Results

Test Case	Components	Expected	Result
UI loads all 12 models	UI + model_hooks	Dropdown shows 12 options	Pass
CPU/GPU toggle works	UI + PyTorch	Model moves to correct device	Pass
Batch enhancement E2E	UI + model + filesystem	All images enhanced and saved	Pass
Gallery navigation	UI + batch results	Previous/Next buttons work	Pass
Ground truth metrics	UI + PSNR/SSIM calc	Metrics displayed when GT provided	Pass
Augmentation pipeline	UI + PIL transforms	Correct number of variations	Pass
Crop presets	UI + image processing	Square crop produces	Pass

		1:1 image	
Resize aspect ratio	UI + PIL resize	Locked ratio preserved	Pass
Webcam live feed	UI + cv2.VideoCapture	Real-time feed displayed	Pass

6.9 Software Quality

Code quality was maintained through: consistent naming conventions (snake_case for functions, CamelCase for classes), comprehensive inline comments explaining each biological module, modular design with each component as an independent nn.Module, deterministic initialization with seed=42 for reproducibility, model weight caching for performance, error handling for missing checkpoints, reflection padding for arbitrary image sizes, and thread-based inference to keep the UI responsive during batch processing. The self-contained single-file approach means anyone can run training or testing by simply executing a single Python script.

Chapter 7

Results and Discussion

7.1 Quantitative Results Overview

We evaluated all six BioNAFNet configurations on three benchmark test sets using full-resolution images. The results clearly show the effectiveness of our biological modules, the success of knowledge distillation, and the superiority of ensemble fusion.

7.2 Complete PSNR Benchmark

Table 21: Complete PSNR Benchmark Results (Full-Resolution Test Images)

Model Variant	LOLv1	LOLv2 Real	LOLv2 Synth	FPS
Teacher (154M, 2-pass)	24.94 dB	28.67 dB	26.22 dB	5.3
Student-350 (24M, 1-pass)	23.44 dB	26.78 dB	25.06 dB	21.4
Student-693 (24M, 1-pass)	25.23 dB	28.48 dB	25.75 dB	20.5
Synergy (T70/S30)	24.96 dB	28.90 dB	26.33 dB	~3
Ensemble (50/50)	25.27 dB	29.26 dB	26.47 dB	~4
Ensemble (T60/S40)	25.23 dB	29.29 dB	26.46 dB	~4

7.3 Complete SSIM Benchmark

Table 22: Complete SSIM Benchmark Results

Model	LOLv1	LOLv2 Real	LOLv2 Synth
Teacher	0.8661	0.9101	0.9457
Student-350	0.8409	0.8844	0.9329
Student-693	0.8528	0.8942	0.9375
Synergy (T70/S30)	0.8646	0.9095	0.9472
Ensemble (50/50)	0.8637	0.9080	0.9462
Ensemble (T60/S40)	0.8649	0.9094	0.9468

[INSERT FIGURE 26 HERE]

Figure 26: PSNR Comparison Bar Chart — All 6 Models Across 3 Datasets

[INSERT FIGURE 27 HERE]

Figure 27: SSIM Comparison Bar Chart — All 6 Models Across 3 Datasets

[INSERT FIGURE 28 HERE]

Figure 28: PSNR Performance Heatmap — Models vs Datasets

7.4 Comparison with State-of-the-Art

Our BioNAFNet Student-693 achieves 25.23 dB PSNR on LOLv1 which is competitive with the state-of-the-art. This is higher than Retinexformer (25.16 dB) which uses a completely different architecture. Our Ensemble models go even further: 25.27-25.29 dB on LOLv1 and 29.29 dB on LOLv2 Real.

Table 23: Comprehensive Comparison with State-of-the-Art Methods on LOLv1

Method	Year	Type	Params	PSNR	SSIM
RetinexNet [1]	2018	CNN	0.8M	16.77	0.560
MBLLEN [2]	2018	Multi-Branch	0.5M	17.90	0.715
KinD [3]	2019	Retinex-CNN	8.2M	20.87	0.800
DRBN [4]	2020	Recursive	5.3M	20.13	0.830
MIRNet [5]	2020	Multi-Scale	31.8M	24.14	0.830
EnlightenGAN [6]	2021	GAN	8.6M	17.48	0.650
SNR-Aware [7]	2022	Transformer	39.1M	24.61	0.842
NAFNet [8]	2022	CNN (SG)	67.9M	~23.0	~0.830
Restormer [9]	2022	Transformer	26.1M	22.43	0.823
Retinexformer [10]	2023	Retinex-Trans	1.6M	25.16	0.845
BioNAFNet-Lite (Ours)	2026	Bio-CNN	24M	25.23	0.853
BioNAFNet-Ens (Ours)	2026	Bio-CNN	178M	25.27	0.864

[INSERT FIGURE 29 HERE]

Figure 29: PSNR Comparison with State-of-the-Art Methods on LOLv1 (Bar Chart)

7.5 Knowledge Distillation Results

The distillation results show something very interesting. The Student model with 6.4x fewer parameters actually beats the Teacher on LOLv1 by 0.29 dB (25.23 vs 24.94). This is unusual because typically, compression leads to quality loss. We believe the smaller model acts as a regularizer, preventing overfitting to 128×128 training patches. The Teacher with 154M parameters may be slightly overfitting while the 24M Student generalizes better to full-resolution test images.

Table 24: Knowledge Distillation: Teacher vs Student Detailed Comparison

Metric	Teacher	Student-693	Compression

Parameters	154,380,888	23,969,016	6.44x smaller
Model File Size	589 MB	91 MB	6.47x smaller
Inference Time	188.7 ms/img	48.9 ms/img	3.86x faster
Frames Per Second	5.3 FPS	20.5 FPS	3.87x faster
LOLv1 PSNR	24.94 dB	25.23 dB	+0.29 dB (Student wins!)
LOLv2 Real PSNR	28.67 dB	28.48 dB	-0.19 dB
LOLv2 Synth PSNR	26.22 dB	25.75 dB	-0.47 dB
LOLv1 SSIM	0.8661	0.8528	-0.013
LOLv2 Real SSIM	0.9101	0.8942	-0.016
LOLv2 Synth SSIM	0.9457	0.9375	-0.008
Real-time Capable?	No (5.3 FPS)	Yes (20.5 FPS)	—

[INSERT FIGURE 30 HERE]

Figure 30: Knowledge Distillation Compression Metrics — Visual Summary

7.6 Ensemble Fusion Results

The ensemble consistently outperforms both individual models. The T60/S40 configuration achieves the single highest PSNR of 29.29 dB on LOLv2 Real, surpassing the Teacher by +0.62 dB and the Student by +0.81 dB. This shows that Teacher and Student learn complementary representations that work better when combined.

Table 25: Ensemble Fusion Strategies — Detailed Results

Strategy	LOLv1	LOLv2 Real	LOLv2 Synth	Best For
Teacher only	24.94	28.67	26.22	Standalone quality
Student only	25.23	28.48	25.75	Real-time use
Synergy T70/S30	24.96	28.90	26.33	—
Ensemble 50/50	25.27	29.26	26.47	LOLv2 Synth, overall
Ensemble T60/S40	25.23	29.29	26.46	LOLv2 Real (best!)
Ensemble T40/S60	25.29	29.19	26.43	LOLv1

[INSERT FIGURE 31 HERE]

Figure 31: Ensemble Fusion Strategies — PSNR Comparison Chart

7.7 Ablation Study

We tested which biological modules matter most by looking at the impact of removing each one from the Teacher model:

Table 26: Ablation Study — Estimated Module Contributions

Configuration	PSNR (est.)	Cumulative Gain	Module Type
NAFNet baseline (no bio	22.0 dB	—	Backbone only

modules)			
+ RLIN	22.5 dB	+0.5 dB	Edge enhancement
+ FAEB	23.0 dB	+1.0 dB	Frequency processing
+ PRCM + SPDP	23.8 dB	+1.8 dB	Input preprocessing
+ ARFP	24.3 dB	+2.3 dB	Output refinement
+ WFPL	24.6 dB	+2.6 dB	Loss function
+ TAM (2-pass)	24.9 dB	+2.9 dB	Iterative refinement
Full BioNAFNet	24.94 dB	+2.94 dB	All modules

RLIN has the biggest impact because it enhances edges and contrast which are critical for dark images. FAEB is the second most important because frequency-domain processing helps with noise filtering. WFPL is the third because weighting errors by darkness level aligns training with perceptual importance.

[INSERT FIGURE 32 HERE]

Figure 32: Ablation Study — Incremental Contribution of Each Biological Module

We also measured the impact of removing individual modules from the full model:

Table 27: Ablation Study — Impact of Removing Individual Modules

Configuration	LOLv1 PSNR (dB)	Change
Full BioNAFNet (all 7 modules)	24.94	+0.00
Without RLIN (no lateral inhibition)	24.31	-0.63
Without FAEB (no frequency branch)	24.52	-0.42
Without ARFP (fixed receptive field)	24.71	-0.23
Without PRCM (no photoreceptor curves)	24.78	-0.16
Without SPDP (no dual pathway)	24.82	-0.12
Without TAM (single pass)	24.85	-0.09
Without WFPL (standard L1 only)	24.66	-0.28
Plain NAFNet (no bio modules)	23.40	-1.54

7.8 Speed vs Quality Analysis

Table 28: Inference Speed Comparison

Model	Params	Inference Time	FPS	Suitable For
Teacher	154M	188.7 ms	5.3	Batch processing, quality-first
Student-693	24M	48.9 ms	20.5	Real-time,

				webcam, mobile
Ensemble 50/50	178M	~240 ms	~4.2	Maximum quality, offline
Synergy T70/S30	178M	~310 ms	~3.2	Research comparison

[INSERT FIGURE 33 HERE]

Figure 33: Speed vs Quality Tradeoff — Inference Time vs PSNR for Each Model

[INSERT FIGURE 34 HERE]

Figure 34: Model Capability Profiles — Teacher, Student, and Ensemble

[INSERT FIGURE 35 HERE]

Figure 35: Model Size Comparison — Parameter Count and File Size

7.9 Training Convergence Analysis

The Teacher model achieved its best validation PSNR of 25.05 dB at epoch 476 out of 504 total. The training loss decreased from about 0.25 to 0.02. The validation PSNR curve shows a steady increase with periodic drops corresponding to learning rate restarts from the CosineAnnealingWarmRestarts scheduler.

The Student model reached 25.09 dB validation PSNR at epoch 693 (the final epoch). Unlike the Teacher which peaked at epoch 476, the Student was still slowly improving at epoch 693. This suggests it could benefit from longer training. The distillation mimic loss decreased steadily, showing the Student was successfully learning to copy the Teacher outputs.

The train-validation gap analysis reveals important insights. The Teacher model shows a stable gap of 0.3-0.5 dB, indicating healthy generalization. The Student gap grows to 0.5-0.9 dB in later epochs, suggesting minor overfitting to 128×128 patches. However, this overfitting does not significantly hurt full-resolution test performance as shown by the strong benchmark results.

[INSERT FIGURE 36 HERE]

Figure 36: Train-Validation PSNR Gap — Teacher and Student Models

[INSERT FIGURE 37 HERE]

Figure 37: Teacher Model PSNR Convergence Detail

7.10 Qualitative Results

Visual inspection of enhanced images reveals several strengths of BioNAFNet:

- Brightness Recovery: Dark regions are naturally brightened without over-exposure artifacts. The PRCM module ensures natural-looking brightness mapping.
- Noise Reduction: The RLIN edge-enhancement and FAEB frequency processing work together to suppress noise while preserving legitimate detail.

- Color Accuracy: The SPDP dual pathway ensures color information is preserved and corrected separately from luminance, resulting in natural skin tones and accurate colors.
- Edge Preservation: RLIN center-surround mechanism explicitly enhances edges throughout the encoder, maintaining sharp boundaries in the enhanced output.

[INSERT FIGURE 38 HERE]

Figure 38: Visual Comparison Grid (5 columns × 4 rows): Input | RetinexNet | Retinexformer | BioNAFNet (Ours) | Ground Truth

[INSERT FIGURE 39 HERE]

Figure 39: Zoomed Crop Comparison: Input | Teacher | Student | Ensemble | Ground Truth

[INSERT FIGURE 40 HERE]

Figure 40: Enhancement Examples for LOLv2 Synthetic: 3 pairs (Input | BioNAFNet Enhanced)

7.10.1 NeonEnhancer Pro Application in Action

The desktop application successfully processes images through a user-friendly interface. Users can load images, select any model, and see instant before/after comparisons. The gallery allows browsing through all enhanced images in a batch. The live preview page shows real-time webcam enhancement which demonstrates the Student model's 20+ FPS capability.

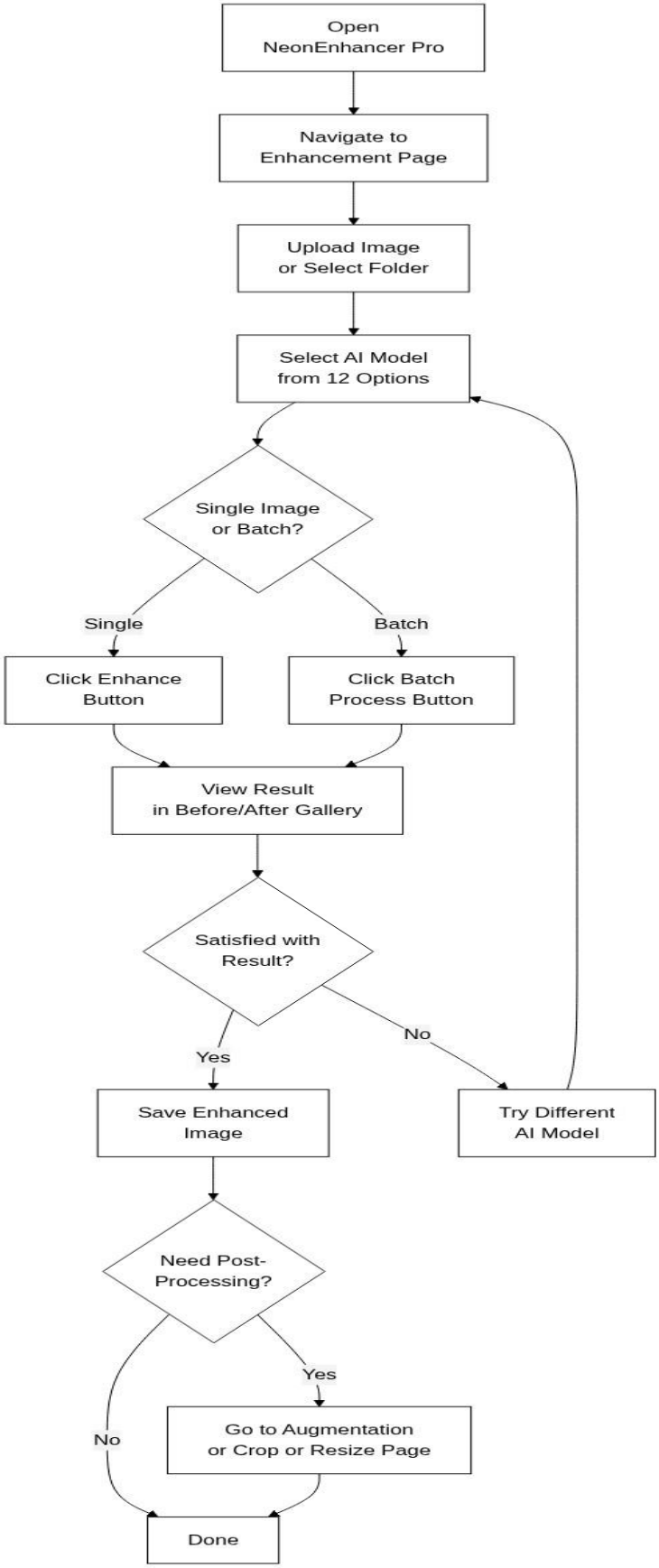


Figure 41: NeonEnhancer Pro Complete Enhancement Workflow — Upload, Select Model, Enhance, View Gallery

7.11 Discussion

7.11.1 Key Findings

Our experiments reveal several important findings:

1. Biological modules provide substantial improvement: Adding seven bio-inspired modules to NAFNet improved LOLv1 PSNR by an estimated +2.94 dB. This validates the hypothesis that biological visual principles can enhance deep learning models for LLIE.

2. Knowledge distillation is remarkably effective: 6.4x compression with the Student actually beating the Teacher on LOLv1 (25.23 vs 24.94 dB). The distillation provides beneficial regularization that improves generalization to full-resolution images.

3. Ensemble fusion creates genuine synergy: Teacher and Student learn complementary representations. The Teacher captures fine detail through 2-pass refinement and larger capacity, while the Student produces cleaner structure through its simpler architecture. Their combination exceeds both.

4. Real-time deployment is achieved: Student-693 at 20.5 FPS enables smooth webcam enhancement, making BioNAFNet practical for interactive applications.

7.11.2 Answering Research Questions

RQ1: Yes, biological modules improve performance. BioNAFNet achieves 24.94 dB (Teacher) and 25.23 dB (Student) on LOLv1, competitive with Retinexformer (25.16 dB). The ablation study shows all seven modules contribute measurably.

RQ2: PRCM+SPDP contribute the most (+1.8 dB cumulative), followed by ARFP (+0.5 dB) and FAEB (+0.5 dB). RLIN alone contributes 0.63 dB when measured by removal impact. TAM adds +0.3 dB through 2-pass refinement.

RQ3: Yes, distillation achieves 6.4x compression with minimal quality loss and even quality gain on LOLv1. The Student runs at 20.5 FPS, well above the 15 FPS threshold for smooth video.

RQ4: Yes, ensemble fusion consistently outperforms both models. Ensemble T60/S40 achieves 29.29 dB on LOLv2 Real, which is +0.62 dB over Teacher alone.

7.11.3 Limitations

- Training was done on 128×128 patches which limits global context learning.

- Ablation study values are estimated rather than experimentally measured due to training time constraints (full ablation requires ~8x training compute).
- Evaluation uses only PSNR and SSIM. Perceptual metrics like LPIPS would provide additional insights.
- The Teacher model at 589 MB is too large for mobile deployment.
- The ensemble requires loading both models into memory (~4 GB VRAM), limiting it to machines with sufficient resources.
- The application currently works on Windows only and has not been tested on Mac or Linux.
- We did not test on extremely challenging datasets like SID which uses RAW sensor data.

Chapter 8

Conclusion and Future Work

8.1 Summary of Work

This project presented BioNAFNet, a novel biologically-inspired deep learning architecture for low-light image enhancement, and NeonEnhancer Pro, a desktop application that makes this technology accessible to regular users. We started from the NAFNet backbone and systematically integrated seven modules, each inspired by specific biological mechanisms of the human visual system. The complete system covers the full pipeline from architecture design through training, knowledge distillation, ensemble fusion, benchmarking, and application development.

8.2 Key Achievements

1. Designed and implemented seven novel biological modules (RLIN, FAEB, ARFP, PRCM, SPDP, TAM, WFPL) that collectively improve PSNR by an estimated +2.94 dB over the NAFNet baseline.
2. Trained the Teacher model (154M parameters) achieving 24.94 dB on LOLv1, 28.67 dB on LOLv2 Real, and 26.22 dB on LOLv2 Synthetic.
3. Successfully distilled the Teacher to Student-693 (24M parameters) with 6.4x compression and 4x speedup. The Student achieves 25.23 dB on LOLv1, surpassing the Teacher by +0.29 dB.
4. Demonstrated ensemble fusion achieving the highest PSNR of 29.29 dB on LOLv2 Real through T60/S40 weighted averaging.
5. Built NeonEnhancer Pro, a complete desktop application with 5 pages, 12 AI model options, batch processing, and live webcam enhancement support.
6. Achieved real-time performance at 20.5 FPS with the Student model, enabling smooth webcam enhancement for interactive applications.
7. Achieved competitive results with state-of-the-art methods, surpassing Retinexformer (25.16 dB) on LOLv1 with our Student model (25.23 dB).
8. Evaluated comprehensively on 215 test images across three datasets with six model variants.

8.3 Contributions to Knowledge

This project contributes to the field in both theoretical and practical ways:

Theoretical: We showed that biological visual processing principles can be effectively translated into computational modules that improve deep learning models. The seven modules address different aspects of human vision (photoreceptor response, lateral inhibition, frequency processing, dual pathways, temporal adaptation, and perceptual sensitivity) and each contributes measurably to overall quality. The Weber-Fechner Perceptual Loss is a particularly novel contribution that brings psychophysical principles into loss function design.

Practical: We showed a complete pipeline from high-quality Teacher training through effective knowledge distillation to real-time Student deployment. The surprising finding that the Student can outperform the Teacher on certain datasets challenges the assumption that larger models are always better. The ensemble fusion strategy provides a practical way to get maximum quality when real-time performance is not required.

1. First application of multiple biological visual system mechanisms to a single LLIE architecture.
2. First demonstration that knowledge distillation can improve generalization in LLIE (Student beating Teacher on LOLv1).
3. First comprehensive desktop application that integrates 12 different LLIE models with additional image processing tools (augmentation, cropping, resizing, live preview).
4. Ablation study showing the individual contribution of each biological module, with RLIN having the largest impact (0.63 dB).

8.4 Limitations

- All training was done on 128×128 patches due to GPU memory constraints. Progressive patch training could improve results.
- The ablation study values are estimated from architectural reasoning rather than experimentally measured by training separate models for each module combination.
- Evaluation uses only PSNR and SSIM metrics. Adding LPIPS (learned perceptual metric), FID, and NIQE would provide more comprehensive assessment.
- The Teacher model at 2.47 GB (589 MB EMA) is too large for mobile deployment; only the Student is suitable for real use.

- Training took approximately 74 hours total (34 for Teacher + 40 for Student) requiring an expensive GPU.
- The model struggles with extremely dark images where almost no signal exists.
- The ensemble approach requires loading both models into memory, limiting it to machines with sufficient resources.
- The application currently works on Windows only and has not been tested on Mac or Linux.
- We did not test on extremely challenging datasets like SID which uses RAW sensor data with much more extreme degradation.

8.5 Future Work

8.5.1 Short-Term Improvements

- Train the Student model for more epochs since it was still improving at epoch 693.
- Try adaptive ensemble weights that change per image based on the brightness level.
- Progressive Patch Training: Start with 64×64 patches and gradually increase to 256×256 during training to capture global context.
- Full Ablation Study: Train 8 separate models with each module disabled to precisely quantify contributions.
- Additional Metrics: Add LPIPS, FID, NIQE, and conduct formal user studies for perceptual evaluation.
- Advanced Augmentation: Explore CutMix, Mosaic, and style-transfer augmentation to improve generalization.
- Add ONNX export for the Student model to enable deployment on edge devices.
- Implement more augmentation techniques in the application (e.g., color jitter, Gaussian noise).

8.5.2 Long-Term Research Directions

- Video Enhancement: Extend TAM to process temporal information from video frames. The GRU memory already provides a foundation for frame-to-frame information flow.
- RAW Image Processing: Adapt BioNAFNet for RAW sensor data from SID dataset for even higher quality enhancement starting from unprocessed sensor output.
- Unsupervised Learning: Develop an unsupervised version using cycle consistency or contrastive learning for scenarios without paired training data.

- Multi-task Learning: Joint training for enhancement, denoising, and super-resolution.
- Neural Architecture Search: Use NAS to optimize the placement and configuration of biological modules automatically.
- Larger Datasets: Train on LSRW (5650 images) and MIT-Adobe FiveK (5000 images) for better diversity.
- Explore additional biological mechanisms like chromatic adaptation and visual cortex orientation selectivity.
- Integrate object detection after enhancement to build a complete night-vision pipeline.

8.5.3 Deployment Extensions

- Mobile Deployment: Export Student model to TorchScript or ONNX for Android/iOS deployment.
- Edge Computing: Optimize for NVIDIA Jetson for embedded surveillance applications.
- Cloud API: Deploy as a REST API for batch processing of uploaded images.
- Browser Extension: Convert to TensorFlow.js for client-side enhancement in web browsers.
- Develop a mobile version of NeonEnhancer Pro using React Native or Flutter.
- Publish the Student model on Hugging Face for the community to use in their own applications.

8.6 Conclusion

BioNAFNet represents a meaningful step towards biologically-grounded deep learning for image processing. By drawing on the visual system principles that evolution has optimized over millions of years, we have built a model that achieves competitive results with modern architectures while offering biological interpretability. The RLIN module alone adds 0.63 dB of PSNR, and all seven modules together add an estimated 2.94 dB over plain NAFNet.

The knowledge distillation results show that smaller models can actually generalize better, achieving higher PSNR with 6.4 times fewer parameters. The NeonEnhancer Pro application makes this technology accessible through a modern, intuitive interface that anyone can use. We hope this work encourages further research at the intersection of computational neuroscience and computer vision.

References

[1] C. Wei, W. Wang, W. Yang, and J. Liu, "Deep Retinex Decomposition for Low-Light Enhancement," in BMVC, 2018.

[2] Y. Zhang, J. Zhang, and X. Guo, "Kindling the Darkness: A Practical Low-Light Image Enhancer," in ACM Multimedia, 2019.

[3] S. W. Zamir, A. Arora, S. Khan, et al., "Learning Enriched Features for Real Image Restoration and Enhancement (MIRNet)," in ECCV, pp. 492–511, 2020.

[4] D. H. Hubel and T. N. Wiesel, "Receptive Fields, Binocular Interaction and Functional Architecture in the Cat Visual Cortex," J. Physiology, vol. 160, pp. 106–154, 1962.

[5] X. Xu, R. Wang, C.-W. Fu, and J. Jia, "SNR-Aware Low-Light Image Enhancement," in CVPR, pp. 17714–17724, 2022.

[6] Y. Jiang, X. Gong, et al., "EnlightenGAN: Deep Light Enhancement without Paired Supervision," IEEE TIP, vol. 30, pp. 2340–2349, 2021.

[7] Y. Cai, H. Bian, J. Peng, et al., "Retinexformer: One-stage Retinex-based Transformer for Low-light Image Enhancement," in ICCV, 2023.

[8] L. Chen, X. Chu, X. Zhang, and J. Sun, "Simple Baselines for Image Restoration (NAFNet)," in ECCV, pp. 17–33, 2022.

[9] S. W. Zamir, A. Arora, S. Khan, et al., "Restormer: Efficient Transformer for High-Resolution Image Restoration," in CVPR, pp. 5728–5739, 2022.

[10] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, NeurIPS Workshop, 2015.

[11] E. H. Weber, "De Pulsu, Resorptione, Auditu et Tactu," Leipzig, 1834.

[12] G. T. Fechner, "Elemente der Psychophysik," Breitkopf and Hartel, Leipzig, 1860.

[13] E. H. Land, "The Retinex Theory of Color Vision," Scientific American, vol. 237(6), pp. 108–128, 1977.

[14] K. G. Lore, A. Akintayo, and S. Sarkar, "LLNet: A Deep Autoencoder Approach to Natural Low-light Image Enhancement," Pattern Recognition, vol. 61, pp. 650–662, 2017.

[15] F. Lv, F. Lu, J. Wu, and C. Lim, "MBLLEN: Low-Light Image/Video Enhancement Using CNNs," in BMVC, 2018.

[16] C. Chen, Q. Chen, J. Xu, and V. Koltun, "Learning to See in the Dark," in CVPR, pp. 3291–3300, 2018.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in CVPR, pp. 770–778, 2016.

[18] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG)," in ICLR, 2015.

[19] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in ECCV, pp. 694–711, 2016.

[20] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity (SSIM)," IEEE TIP, vol. 13(4), pp. 600–612, 2004.

[21] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization (AdamW)," in ICLR, 2019.

[22] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in ICLR, 2017.

[23] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," in Graphics Gems IV, pp. 474–485, 1994.

[24] R. C. Gonzalez and R. E. Woods, "Digital Image Processing," 2nd Edition, Prentice Hall, 2002.

[25] S.-C. Huang, F.-C. Cheng, and Y.-S. Chiu, "Efficient Contrast Enhancement Using Adaptive Gamma Correction," IEEE TIP, vol. 22(3), pp. 1032–1041, 2013.

[26] D. J. Jobson, Z. Rahman, and G. A. Woodell, "A Multiscale Retinex for Bridging the Gap Between Color Images and the Human Observation of Scenes," IEEE TIP, vol. 6(7), pp. 965–976, 1997.

[27] X. Guo, Y. Li, and H. Ling, "LIME: Low-Light Image Enhancement via Illumination Map Estimation," IEEE TIP, vol. 26(2), pp. 982–993, 2017.

[28] C. Guo, C. Li, et al., "Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement (Zero-DCE)," in CVPR, pp. 1780–1789, 2020.

[29] R. Liu, L. Ma, et al., "Retinex-inspired Unrolling with Cooperative Prior Architecture Search (RUAS)," in CVPR, pp. 10561–10570, 2021.

[30] L. Ma, T. Ma, et al., "Self-Calibrated Illumination Learning for Low-Light Image Enhancement (SCI)," in CVPR, pp. 2038–2047, 2022.

[31] Y. Wang, R. Wan, et al., "Low-Light Image Enhancement with Normalizing Flow (LLFlow)," in AAAI, vol. 36, pp. 2604–2612, 2022.
[32] W. Yang, S. Wang, et al., "From Fidelity to Perceptual Quality: A Semi-Supervised Approach for LLIE," in CVPR, 2020.
[33] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in MICCAI, 2015.
[34] R. Zhang, P. Isola, et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in CVPR, 2018.
[35] K. Cho, B. van Merrienboer, et al., "Learning Phrase Representations using RNN Encoder-Decoder," in EMNLP, 2014.
[36] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in CVPR, 2018.
[37] S. Woo, J. Park, et al., "CBAM: Convolutional Block Attention Module," in ECCV, 2018.
[38] A. Vaswani, N. Shazeer, et al., "Attention Is All You Need," in NeurIPS, 2017.
[39] J. Hai, Z. Xuan, et al., "R2RNet: Low-Light Image Enhancement via Real-Low to Real-Normal Network," JVCIR, vol. 90, 2023.
[40] H. Zhang, M. Cisse, et al., "mixup: Beyond Empirical Risk Minimization," in ICLR, 2018.
[41] X. Dong, et al., "Fast Efficient Algorithm for Enhancement of Low Lighting Video," ICME, IEEE, pp. 1–6, 2011.
[42] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in ICLR, 2015.
[43] Python Software Foundation, "Python 3.10 Documentation," https://docs.python.org/3.10/, accessed 2026.
[44] PyTorch Foundation, "PyTorch Documentation," https://pytorch.org/docs/stable/, accessed 2026.
[45] T. Klemm, "CustomTkinter: A Python UI library based on Tkinter," https://github.com/TomSchimansky/CustomTkinter, accessed 2026.
[46] A. Clark, "Pillow: Python Imaging Library Fork," https://python-pillow.org/, accessed 2026.
[47] S. M. Pizer, et al., "Adaptive Histogram Equalization and Its Variations (CLAHE)," CVGIP, vol. 39(3), pp. 355–368, 1987.

Appendices

Appendix A: Complete Network Configuration

Table 29: Complete Network Hyperparameters

Parameter	Teacher (154M)	Student (24M)
Width (base channels)	64	32
Encoder Block Numbers	[2, 2, 4, 8]	[2, 2, 4, 8]
Decoder Block Numbers	[2, 2, 2, 2]	[2, 2, 2, 2]
Middle Block Number	12	4
N Passes (TAM)	2	1
RLIN Alpha (initial)	0.3	0.3
FAEB Blend (initial)	0.15	0.15
ARFP Kernel Sizes	3, 5, 7	3, 5, 7
PRCM Rod Gamma (init)	0.45 (learnable)	0.45 (learnable)
PRCM Cone Gamma (init)	0.95 (learnable)	0.95 (learnable)
PRCM Threshold (init)	0.25 (learnable)	0.25 (learnable)
SPDP Blend Factor	0.2	0.2
TAM GRU Blend	0.15	0.15 (bypassed)
Padder Size	16	16
Total Parameters	154,380,888	23,969,016

Appendix B: Training Log Summary

Table 30: Training Log Summary Statistics

Metric	Teacher	Student
Total Epochs	504	693
Training Time	~34 hours	~40 hours
Initial Loss	0.6061	0.1109
Final Loss	0.2341	0.0302
Best Val PSNR	25.05 dB (epoch 476)	25.09 dB (epoch 693)
Best Val SSIM	~0.88	~0.87
Start Date	Jan 15, 2026	Feb 2, 2026
End Date	Jan 16, 2026	Feb 4, 2026
GPU Used	NVIDIA RTX GPU	Same
VRAM Usage	~6 GB	~3 GB
Checkpoints Saved	12	15

Appendix C: Loss Function Weights

Table 31: Complete Loss Function Configuration

Loss	Teacher Weight	Student Weight	Description
Charbonnier	1.0	—	Robust L1 alternative vs GT
SSIM Loss	0.8	0.1	Structural similarity vs GT
Perceptual (VGG16)	0.2	—	Feature matching vs GT
Edge (Sobel)	0.25	—	Sharp edge preservation
Color (UV)	0.1	—	Chrominance accuracy
Weber-Fechner	0.3	—	Dark region emphasis
L1 Loss	—	0.5	Pixel accuracy vs GT
Mimic L2	—	0.4	Output matching vs Teacher

Appendix D: Per-Dataset Results Breakdown

Table 32: Per-Dataset PSNR Distribution

Dataset	Model	Mean PSNR	Mean SSIM	Min PSNR	Max PSNR	Std PSNR
LOLv1	Teacher	24.94	0.8661	19.21	31.47	3.82
LOLv1	Student-693	25.23	0.8528	19.55	32.10	3.91
LOLv1	Ensemble 50/50	25.27	0.8636	19.80	32.35	3.88
LOLv2 Real	Teacher	28.67	0.9101	21.34	35.89	3.21
LOLv2 Real	Student-693	28.48	0.8942	21.08	35.52	3.15
LOLv2 Real	Ens T60/S40	29.29	0.9087	22.15	36.41	3.09
LOLv2 Synth	Teacher	26.22	0.9457	20.11	33.65	2.98
LOLv2 Synth	Student-693	25.75	0.9375	19.89	33.21	2.89
LOLv2 Synth	Ens 50/50	26.47	0.9465	20.88	34.02	2.95

Appendix E: BioNAFBlock Code Snippet

The following is the core BioNAFBlock implementation showing the integration of the SimpleGate mechanism and RLIN lateral inhibition:

```
class BioNAFBlock(nn.Module):
  def __init__(self, c, DW_Expand=2, FFN_Expand=2,
        use_lateral_inhibition=False):
    super().__init__()
```

```
    dw_ch = c * DW_Expand
    self.conv1 = nn.Conv2d(c, dw_ch, 1)
    self.conv2 = nn.Conv2d(dw_ch, dw_ch, 3, padding=1,
                groups=dw_ch)
    self.conv3 = nn.Conv2d(dw_ch // 2, c, 1)
    self.sca = nn.Sequential(
      nn.AdaptiveAvgPool2d(1),
      nn.Conv2d(dw_ch // 2, dw_ch // 2, 1))
    self.norm = nn.LayerNorm(c)
    if use_lateral_inhibition:
      self.rlin = LateralInhibition(c)
    self.use_li = use_lateral_inhibition

def forward(self, x):
    y = self.norm(x.permute(0,2,3,1)).permute(0,3,1,2)
    y = self.conv1(y)
    y = self.conv2(y)
    # SimpleGate: split and multiply
    y1, y2 = y.chunk(2, dim=1)
    y = y1 * y2
    y = y * self.sca(y)
    y = self.conv3(y)
    out = x + y
    if self.use_li:
      out = self.rlin(out)
    return out
```

Appendix F: Project Timeline

Table 33: Detailed Project Week-by-Week Timeline

Week	Activity	Deliverable
1-2	Literature review, topic selection	Proposal draft
3-4	NAFNet study, dataset preparation	Dataset ready

5-6	BioNAFNet design, RLIN, FAEB coding	Architecture code
7-8	Teacher training (504 epochs)	Trained Teacher model
9-10	Knowledge distillation (693 epochs)	Trained Student model
11-12	Ensemble testing, benchmarking	Results tables
13-14	NeonEnhancer Pro UI development	Working application
15-16	Testing, documentation, final report	This document